

UNIVERSIDAD COMPLUTENSE DE MADRID

FACULTAD DE MATEMÁTICAS



Máster en Ingeniería Matemática

TRABAJO DE FIN DE MÁSTER

SIMULACIÓN DE UN MODELO ESPACIAL DE EVOLUCIÓN TUMORAL CON R Y C++

Autor: Alberto Parramón Castillo

Director: Ramón Díaz Uriarte

Tutor: Gerardo Enrique Oleaga

Septiembre 2018

SIMULACIÓN DE UN MODELO ESPACIAL DE EVOLUCIÓN TUMORAL CON R Y C++

Autor: Alberto Parramón Castillo

Director: Ramón Díaz Uriarte

Tutor: Gerardo Enrique Oleaga

Facultad de Matemáticas

Universidad Complutense de Madrid

Septiembre 2018

Índice general

Índice de Figuras	v
Índice de Tablas	vi
1. Introducción	1
1.1. Motivación del proyecto	1
1.2. Objetivos y enfoque	2
2. Estado del arte	3
2.1. Modelos de crecimiento tumoral	3
2.2. Modelos espaciales	5
2.2.1. Modelo espacial de Waclaw	5
2.2.2. Modelo espacial de Sottoriva	7
2.2.3. Modelo espacial de Sun	8
2.2.4. Características, conceptos y conclusiones	9
2.3. Paquete OncoSimulR	10
2.3.1. Modelo en tiempo continuo de OncoSimulR: algoritmo de Mather	14
3. Diseño y desarrollo	17
3.1. Algoritmo	17
3.1.1. Detalle de las cuatro fases del algoritmo	18
3.1.2. Parámetros de entrada y de salida	19
3.1.3. Otros aspectos: estructura de vecindario, tamaño del deme y simulación gráfica	20
3.1.4. Modificación del algoritmo y modelo de OncoSimulR para usar demes de población aproximadamente constante	22
3.1.5. Resumen de características	23
4. Instalación, ejecución, pruebas y resultados	25
4.1. Instalación	25
4.2. Ejecución y resultados	26
4.2.1. Visualización de resultados en un escenario clásico	31
4.3. Rendimiento	35

5. Conclusiones y líneas de trabajo futuro	39
Bibliografía	42

Índice de Figuras

2.1. Visualización del tumor en forma de bolas para diferentes valores de M	6
2.2. Grafo de ejemplo	13
3.1. Estructuras espaciales de vecinos	21
4.1. Evolución tumoral: Primeras 15 iteraciones	32
4.2. Evolución tumoral: iteraciones 15-45	32
4.3. Evolución tumoral: situacion final	33
4.4. Situación final migracion larga distancia $1e-6$	34
4.5. Situación final migracion larga distancia $1e-4$	34
4.6. Situación final migracion larga distancia $1e-3$	35

Índice de Tablas

2.1. Características principales de los modelos espaciales discutidos	10
4.1. Resultados rendimiento de tiempos	36
4.2. Resultados rendimiento de memoria	37

1

Introducción

En este capítulo se detallan las necesidades que dan lugar a la realización de este Trabajo de Fin de Máster, planteando los objetivos que habrán de cumplirse para llevarlo a cabo. Al final del capítulo se explicará la estructura de este documento.

1.1. Motivación del proyecto

El cáncer se encuentra cada vez más presente en nuestro día a día. El temor a padecerlo es un hecho que la mayoría de las personas experimenta en algún momento de su vida, ya sea por la proximidad con familiares o amigos que lo han tenido, o por alcanzar edades en las que la frecuencia de estas enfermedades es cada vez mayor.

Pese a los numerosos avances que se han producido en los últimos años en su diagnóstico, tratamiento y prevención, el término cáncer está intimamente asociado a la muerte por la mayoría de la población. El número de diagnósticos que se producen cada año aumenta, lo que indica que cada vez es más probable que cualquiera pueda llegar a sufrir algún tipo de cáncer a lo largo de su vida.

Pese a que los tipos de cánceres son distintos unos de otros, todos tienen algo en común: son debidos al crecimiento anormal de un conjunto de células. El cáncer surge porque en algún momento se produce en la división celular una mutación que provoca que las células se dividan de forma descontrolada y el cuerpo no sea capaz de controlarlo, formándose así una masa tumoral.

Pese a que cada vez se diagnostican más casos de cáncer, también cada vez aumenta más la esperanza de vida y de cura para las personas que padecen esta enfermedad. Según el Informe Anual a la Nación ([1]), los índices de mortalidad por cáncer están descendiendo. El informe declara que los índices de mortalidad durante el período de 2010 a 2014 disminuyeron en 11 de los 16 tipos de cáncer más comunes en los hombres y en 13 de los 18 tipos más comunes de cáncer en las mujeres, incluidos el cáncer de pulmón, el de colon y recto, de seno femenino y el de próstata.

Por todo esto, y con el objetivo de aportar su granito de arena en la lucha contra el cáncer, surgen los modelos matemáticos sobre la replicación y mutación celular, es decir, sobre el crecimiento tumoral.

La modelización matemática se ha vuelto cada vez más abundante en la investigación sobre el cáncer. La complejidad del cáncer se adapta bien a estos modelos, que ayudan a dilucidar los mecanismos y a proporcionar predicciones cuantitativas sobre la expansión tumoral. Estos modelos intentan obtener respuestas a la causa de la iniciación del tumor, su progresión y su metástasis. Así como a la respuesta a tratamientos y la resistencia de las mutaciones a los mismos. Los modelos matemáticos no solo permiten complementar los estudios experimentales y clínicos sino también desafiar paradigmas actuales, redefinir nuestra comprensión de los mecanismos tumorales y dar forma a las futuras investigaciones en la biología del cáncer.

El poder del modelado matemático reside en su capacidad para revelar conocimientos previamente desconocidos o contrarios a la intuición, que podrían haber pasado desapercibidos por un enfoque cualitativo más propio de la biología. Los métodos de simulación son muy útiles para examinar el resultado de escenarios que son intratables matemáticamente. Mediante el muestreo de los datos que producen y el análisis estadístico de los resultados se puede predecir el comportamiento de la población ante un escenario concreto de una forma sencilla y rápida. En concreto, los modelos espaciales de crecimiento tumoral están revolucionando esta línea de la investigación, permitiendo simular situaciones mucho más realistas.

1.2. Objetivos y enfoque

Los objetivos que se quieren conseguir al terminar este proyecto son:

- Entender la utilidad y el funcionamiento de los modelos espaciales de simulación tumoral, por encima del resto de modelos.
- Conocer en profundidad las características de los modelos espaciales.
- Implementar un modelo espacial competitivo, que esté a la altura o por encima de los ya existentes.
- Integrar este modelo en el paquete OncoSimulR de R, haciendo uso de la funcionalidad que dicho paquete ofrece.

Es importante señalar lo que este proyecto **no** incluye, ya que este es solo una parte de un trabajo de investigación que llevo realizando con el profesor Ramón Díaz Uriarte tiempo atrás. Por tanto, no incluye el tiempo dedicado a comprender la terminología básica de los modelos de crecimiento tumoral, o el funcionamiento básico del paquete OncoSimulR. Pero **sí** incluye el tiempo dedicado a entender los modelos de crecimiento **espacial**. Aún así, en este documento se dedicará una parte de la sección 'Estado del arte' a contar estos conceptos para que un lector que no este familiarizado con la rama pueda seguir el resto de la lectura.

Se va a iniciar el documento echando un vistazo al estado del arte, es decir, a todo lo que ya hay investigado sobre el tema y la base de la que se parte. A continuación se explicará el algoritmo que se ha diseñado, se comprobarán los resultados que ofrece así como su rendimiento. Finalmente se acabará exponiendo un resumen del trabajo realizado, las conclusiones y las líneas de trabajo futuro.

2

Estado del arte

En este capítulo se van a definir los principales conceptos que aparecen en los modelos de crecimiento tumoral. Se van a explicar brevemente los distintos tipos de modelos que hay y con más detalle algunos ejemplos de modelos espaciales. Finalmente se hablará del paquete *OncoSimulR* sobre el cual se implementará, en el siguiente capítulo, el nuevo algoritmo.

2.1. Modelos de crecimiento tumoral

En el contexto de esta memoria, un modelo de evolución tumoral tiene como objetivo simular lo que ocurre a lo largo del tiempo en las células que componen un tumor, incluyendo la aparición y desaparición de distintos clones de células como resultado de procesos de mutación y selección.

El aspecto más crítico en la aparición de cáncer es la mutación de las células. Una mutación puede provocar a esa célula una ventaja evolutiva frente a las demás, o por el contrario, una desventaja. En la primera de las situaciones, lo más probable es que con el paso del tiempo esta célula se reproduzca y que sus genes se hagan cada vez más abundantes en la población, imponiéndose sobre los demás. En el segundo de los casos, se espera todo lo contrario, es decir, que las células que experimenten esa mutación terminen por desaparecer.

Dicho esto, es importante definir algunos términos comúnmente utilizados [2] [3]:

- Genotipo: se define como genotipo al estado (mutado o sin mutar) del conjunto de los genes que tienen las células.
- Fitness o aptitud: se suele definir como la contribución reproductiva de un genotipo a la siguiente generación. Suele ser un número que se le asocia a un genotipo, a mayor valor, más ventajoso es el genotipo, esto es, la célula será resistente y tendrá mayor capacidad de reproducción. Las células con mayor fitness son las células que provocan el crecimiento tumoral y la expansión del tumor.
- Mutaciones pasajeras (*Passenger mutations*): son cambios genéticos que no están directamente relacionados con el desarrollo del cáncer, ya que no influyen positivamente en el fitness.

- Mutaciones conductoras (*Driver mutations*): son cambios genéticos que están muy relacionados con el desarrollo del cáncer. Típicamente producen una ventaja evolutiva y un aumento del fitness.
- Fijación: una mutación se ha fijado cuando su frecuencia en la población es 1.
- Eventos celulares: la mutación, la reproducción y la muerte de células son ejemplos de eventos celulares.

Se distinguen distintos tipos de modelos de crecimiento. A continuación se van a comentar algunos de ellos.

En función de cómo se trate el tiempo:

- Modelo en tiempo discreto [4] [5] [6] [7] [8] : cuando solo se muestrean los valores de salida en un conjunto discreto (de cardinal finito o numerable) de instantes de tiempo. Llevado a este campo, cuando se usan estos modelos lo que sucede es que en cada instante de tiempo se pueden producir uno, ninguno o varios eventos.
- Modelo en tiempo continuo [9] [10]: si se quieren conocer los valores de salida en todos los instantes de un intervalo de tiempo. Los modelos continuos suelen estar basados en ecuaciones diferenciales, tanto ordinarias como en derivadas parciales. Llevado a este campo, normalmente estos modelos lo que hacen es calcular el siguiente instante de tiempo en el que se va a producir algún evento.

En función de los factores de los que dependen los eventos de mutación, reproducción o muerte se pueden distinguir:

- Modelos en los eventos celulares no se influyen de factores microambientales.
- Modelos en los que los eventos celulares sólo se ven influidos por factores microambientales y del entorno que les rodea, por ejemplo, de otras células cercanas
- Modelos híbridos: aquellos que tienen en cuenta ambas diferenciaciones.

En función de cómo avanza el algoritmo en cada iteración [5]. Para este caso se va a asumir una población inicial de N individuos y se van a considerar sólo los procesos de muerte y reproducción.

- Modelo Wright-Fisher (*Wright-Fisher model*): estos modelos asumen que todos los individuos de la población mueren en cada generación y son remplazados por una descendencia de exactamente N individuos. El genotipo de estos nuevos N individuos vendrá dado en función del fitness del genotipo de sus antecesores, de manera que los individuos con más fitness se reproducirán más que los de menos fitness. El aspecto más relevante de estos modelos es que el tamaño se mantiene constante.
- Modelo de Moran (*Moran model*): estos modelos se caracterizan porque en cada iteración un solo individuo es elegido al azar para reproducirse, y otro para morir. Por tanto la población se mantiene constante.
- *Branching Process Model*: según estos modelos, en cada iteración, cada individuo de la población tiene una probabilidad de reproducirse o de morir. En estos modelos se considera que la descendencia de cada célula es independiente de las demás. Se asume que cada individuo con un fitness w_i se reproduce siguiendo una distribución de Poisson de parámetro proporcional al fitness. Para ver cómo fluctúa la población se van a distinguir dos versiones de este modelo:

1. Branching Process Model (clásico): en este modelo la población puede crecer indefinidamente o incluso decrecer hasta extinguirse. Depende de lo que el azar dado por la distribución de Poisson dictamine en cada caso.
2. Logistic Branching Process Model: esta versión del Branching Process Model clásico añade un parámetro para regular el tamaño de la población, de manera que esta se mantenga cercana a la población inicial.

En función del espacio dimensional en el que se trabaje:

- Modelos adimensionales: se considera un conjunto de N individuos y no se diferencia la posición que ocupan unos y otros.
- Modelos espaciales [11] [12]: se consideran modelos espaciales aquellos que están representados en 1, 2 o 3 dimensiones. Cada individuo, aparte de caracterizarse por su genotipo, tiene asociada una posición única, además suelen tener la posibilidad de moverse a espacios no ocupados por otras células. Estos modelos son por tanto mucho más complejos.

La mayor parte de los modelos suelen considerar un genotipo, que está formado por un conjunto de genes, cada uno de los cuales tiene una probabilidad concreta de mutar cuando es evaluado. Se distinguen dos casos:

- Mutación simple: cuando sólo se permite una mutación por iteración. Esto suele ser muy habitual en los modelos en tiempo continuo, ya que el modelo en tiempo continuo se caracteriza por calcular el instante en el que se producirá el siguiente evento, y dos mutaciones serían consideradas como dos eventos, por lo que se harían en iteraciones distintas.
- Mutación múltiple: cuando se permite la mutación de varios genes por iteración. Lo habitual en modelos en tiempo discreto es que al evaluar al individuo también se compruebe si se muta alguno o varios de sus genes o no.

2.2. Modelos espaciales

Se va a dedicar esta sección a ver con algo más de detalle algunos ejemplos que ilustran las características definitorias de los tipo de modelos espaciales más importantes. Se centrará la atención en destacar los puntos fuertes del algoritmo y los detalles en los que cada investigador a querido profundizar. Al final de esta sección seremos capaces de visualizar cuál es el funcionamiento general de un modelo espacial, las fases por las que pasa o los detalles que se deben tener en cuenta al diseñar el algoritmo.

2.2.1. Modelo espacial de Waclaw

El modelo espacial de Waclaw [12] es un modelo de tiempo discreto en 3 dimensiones. Cada célula tiene asociada una posición en el espacio, además se considera que tendrán una capacidad de reproducirse proporcional al número de posiciones vacías que haya a su alrededor, aparte del fitness. Por tanto, se espera que la masa tumoral adquiera forma de bola más o menos esférica y que la reproducción sea mayor en la superficie de la misma.

Como en otros modelos, las células tendrán una probabilidad d de morir, de manera que dejarían un hueco vacío en la bola. Además, este modelo incorpora una probabilidad M de que las células se muevan tras la replicación. Si $M = 0$ la forma del tumor tendrá forma de bola, y si $M > 0$ se parecerá a un conglomerado de bolas. Esto permite sacar otra conclusión: tal

y como se ha definido la probabilidad de replicación, se puede decir que a mayor valor de M mayor velocidad de crecimiento y expansión del tumor.

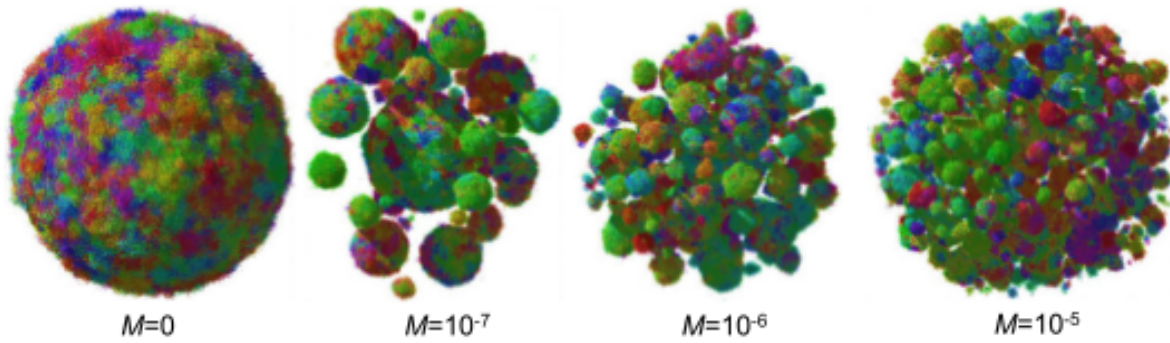


Figura 2.1: Visualización del tumor en forma de bolas para diferentes valores de M (Waclaw et al [12]).

Una característica importante de los modelos es su flexibilidad para poder simular diferentes escenarios. Esto se puede conseguir introduciendo distintos parámetros en el algoritmo informático. Este modelo incorpora la posibilidad de simular varias situaciones interesantes.

Se pueden considerar dos situaciones iniciales distintas:

1. Cuando ya hay un grupo de células con una ventaja genética (y por tanto mayor fitness que las demás) en situación de metástasis, es decir, que ya se han separado del tumor inicial y se disponen a invadir otras zonas del cuerpo.

En esta primera situación inicial se considera que las nuevas alteraciones genéticas que se produzcan no mejoran el fitness (mutaciones pasajeras). Esto se hace con la intención de asemejarse a lo que ocurre realmente: cuando un tumor se ha desarrollado y un conjunto de células con un genotipo determinado se ha impuesto sobre los demás, es porque este genotipo ha dado a esas células una ventaja genética lo suficientemente importante. Por tanto, al moverse a otra zona del cuerpo en la que no hay mutaciones, es natural pensar que este genotipo va a dominar sin demasiados problemas.

2. Cuando se empieza en el tumor inicial: en esta situación se elimina la consideración anterior de que las alteraciones genéticas no modifican el fitness. Por tanto es un estado más complejo ya que las células tumorales están adquiriendo constantemente mutaciones conductoras (*driver mutations*).

También se pueden cambiar otros parámetros para generar escenarios alternativos:

1. Se puede indicar que las células se replicarán con una probabilidad constante en cuanto que haya una sola posición vacía a su alrededor.
2. Se puede indicar que cualquier célula puede replicarse aunque no tenga posiciones vacías a su alrededor, en ese caso la célula replicada empujaría a las demás para hacerse hueco, a este fenómeno se le denomina *pushing*.
3. Se puede indicar que la replicación y la muerte ocurran sólo en la superficie de la bola. Esto provocará mayor heterogeneidad de células, ya que al no haber renovación celular no habrá opción de que las células de mayor fitness se impongan a las demás.

El algoritmo informático que formaliza y simula este modelo (en su escenario básico) es el siguiente:

Por cada instante de tiempo:

1. Se elige una célula i al azar. A su genotipo se le denomina g .
2. Una zona alrededor de la célula i es elegida al azar y si esta vacía, la célula i se replicará y creará una nueva célula j con probabilidad proporcional a la tasa de replicación de la célula i dividido por la máxima tasa de replicación de las células presentes en el tumor: $\frac{b_g}{b_{max}}$. Al depender de que la zona elegida este vacía, también es proporcional al número de posiciones vacías alrededor de i . En caso de no realizarse la replicación se va al paso 5.
3. Cada una de las dos células i y j , reciben una cantidad n_i y n_j de nuevas mutaciones (número elegido al azar a través de una distribución de Poisson). Si n_i es 0, la célula i no cambiará su genotipo. Igual ocurrirá con la célula j si n_j es 0.
4. Con probabilidad M , la nueva célula j se moverá y formará una nueva bola en una zona cercana a la bola de la que procede (la de la célula i). La nueva posición en la que se queda la nueva bola creada tendrá su centro en $X_j = (X_{1,j}, X_{2,j}, X_{3,j})$ (X_j denota el centro de la bola que tiene, entre otras, a j como célula que la compone) y se calculará: $X_j = X_i + R_i \frac{x_i}{|x_i|}$ donde X_i denota la posición del centro de la bola de la célula i original, R_i es el radio de la bola, y x_i es la posición de la célula i relativa al centro de la bola X_i .
5. La célula i muere con probabilidad igual a su tasa de muerte dividido por la tasa de muerte máxima de las células que componen el tumor: $\frac{d_g}{d_{max}}$. Si todas las células de una bola mueren, la bola es eliminada.
6. El tiempo es incrementado $dt = \frac{1}{b_{max}N}$ donde N es el número de células del tumor.

Se trata por tanto de un modelo en tiempo discreto un tanto especial ya que el incremento de tiempo no es constante. Este modelo se puede clasificar como un modelo híbrido ya que los eventos celulares dependen tanto de los factores microambientales (las posiciones vacías alrededor de la célula) como del genotipo de la propia célula.

2.2.2. Modelo espacial de Sottoriva

El modelo de Sottoriva ([13]) tiene como principal objetivo conocer los orígenes del tumor. Sus estudios se centran en el cáncer colorectal, contando además con una base de datos con información (imágenes, documentación) sobre la evolución de este tumor en varios pacientes.

Su idea base es la siguiente: las mutaciones que se hayan generado al comienzo del tumor (y que no hayan causado una desventaja genética) perdurarán en las células en el tumor final; mientras que los genes mutados durante etapas intermedias del crecimiento del tumor aparecerán en un menor porcentaje de células al final del mismo. De esta manera sería fácil conocer cuáles han sido las primeras mutaciones sufridas por las células durante la evolución del tumor.

El algoritmo planteado por Sottoriva comienza con un deme (conjunto de células) formado por 10000 células, en la posición (400, 400, 400). Asume que todas las células dentro del deme tienen el mismo fitness, y que los cambios de fitness ocurren a nivel de deme, como resultado de la obtención de alteraciones somáticas. Estas alteraciones dependen de los parámetros de entrada: probabilidad de mutación μ y magnitud de los cambios de fitness σ . Cuando se produce una mutación en un deme, se modifica su genotipo y se produce un cambio en su fitness. El genotipo viene representado como una cadena de genes, cada gen tiene un identificador y lleva

asociada una cadena binario donde el 1 o el 0 indican si está mutado o no en cada deme. El cambio de fitness provocado por una mutación de un gen viene dado por una distribución Gaussiana de media 0 y desviación típica σ . En cada iteración, un deme se reproduce o muere según una probabilidad proporcional al fitness del deme. Al reproducirse un deme ocupará una nueva posición en el espacio. De este modo los demes con mayor fitness tenderán a expandirse en el tumor. Si un deme que esta completamente rodeado por otros demes se reproduce, el deme hijo empujara a alguno de sus demes vecinos para abrirse hueco en el espacio.

Para tratar de demostrar su hipótesis Sottoriva va muestreando varios demes a lo largo de la evolución del tumor, estudiando en cada instante de tiempo los cambios en su genotipo. Lo que se encuentra al realizar el estudio ratifica su teoría, además también se observa que la mayoría de las mutaciones no dominantes (que no producen una ventaja genética) que suceden en momentos tempranos del tumor se mantienen en las células finales. De aquí viene el nombre de modelo del Big Bang, atribuido a estos modelos que buscan el origen del tumor y que defienden esta teoría. El resultado final es una gran heterogeneidad intratumoral, esto es un conjunto de células con variedad de genotipos diferentes, pero entre los que se encuentra un patrón significativo: las alteraciones genéticas que comparten un gran porcentaje de células son aquellas que han sido producidas en las etapas iniciales del tumor. De esta manera se puede estudiar cómo ha sido la evolución del tumor en sus primeras etapas.

Por último, es importante mencionar que el modelo de Sottoriva toca un aspecto que puede ser muy relevante en los modelos de los próximos años: el papel de los factores ambientales en el tumor. Es decir, cómo afecta al crecimiento del tumor los factores microambientales que hay en la zona donde se encuentra. Sottoriva realiza un pequeño estudio introduciendo algún factor ambiental básico y no obtiene diferencias en los resultados, por lo que concluye que estos no afectan el crecimiento del tumor. Pero indica que sería interesante probar con factores microambientales más complejos, en los que incluso las mismas células pudieran modificar este ambiente. Por tanto, se abre aquí un debate: la importancia del factor ambiental en el desarrollo del tumor.

2.2.3. Modelo espacial de Sun

El modelo de Sun ([14]) sigue líneas parecidas al de Sottoriva. Investiga también tumores colorectales, donde corrobora la dinámica del Big Bang, es decir, la teoría de que la mayoría de las alteraciones subclonales detectables se presentan temprano durante el crecimiento del tumor. Sun desarrolla más el tema de la deriva genética, fenómeno por el cuál las células se mueven y colonizan otras zonas en el espacio, imponiendo su genotipo. Atribuye a este factor la gran heterogeneidad intratumoral presente en los tumores finales. Además Sun asume algo más importante: la evolución solo se produce en los bordes del tumor. El modelo crecimiento periférico es apoyado por estudios recientes indicando que las células cancerígenas en la periferia exhiben una mayor actividad de proliferación que las del núcleo ([15]), de hecho el modelo de Waclaw también contó con esta posibilidad en sus posibles escenarios de simulación ([14]).

El algoritmo que implementa este modelo parte de un espacio tridimensional en forma de cubo de lado $2 \cdot x + 1$, en el que en cada posición puede vivir un conjunto de 1000 o más células (deme). El algoritmo comienza con un deme inicial, con células sin mutar, colocado en el centro del cubo. Este deme se desarrolla y crece hasta alcanzar un tamaño máximo (tam_{max}) predefinido.

Dentro del deme la evolución de las células no es espacial, es adimensional, en tiempo discreto. Eligen un conjunto de células que se van a reproducir, y al reproducirse algunas reciben mutaciones, pudiendo caer más de una mutación sobre la misma célula (mutación múltiple). Las células que no se han reproducido mueren. Los parámetros se eligen de tal manera que el tumor

crezca (las células que se reproducen son en media más de la mitad); cuando el tumor llega al tamaño máximo se pasa a la siguiente fase del algoritmo.

En esta segunda fase es donde entra la componente espacial, se inicia una fase en la que los demes pueden reproducirse. Sun limita la reproducción a aquellos demes que tienen al menos un espacio vacío a su alrededor. En cada iteración se intentarán reproducir tantos demes como espacios habitables haya, esto es, posiciones adyacentes a los demes actuales que estén vacías. Se escogerán al azar esta cantidad de demes, y se reproducirán con una probabilidad predefinida y que también depende de la cantidad de espacios vacíos que haya a su alrededor, la probabilidad será 0 si el deme escogido al azar no tiene posiciones vacías contiguas a él. Para realizar la reproducción lo que se hace es crecer el deme elegido hasta un tamaño igual a $2 * tam_{max}$. Entonces se divide y ocupa un espacio adyacente.

En este algoritmo el tumor siempre crece, dependiendo la velocidad de crecimiento del parámetro de reproducción de las células del deme. Las mutaciones que se producen dentro del deme provocan una competencia entre especies dentro del mismo. Esta competencia no existe entre demes pues una vez que un deme se ha reproducido y ha ocupado una posición en el espacio, no puede desaparecer ni recibir mutaciones del exterior. Además los demes que no estén en la superficie (que tengan todas sus posiciones adyacentes ocupadas) permanecen inmutables en el tiempo.

2.2.4. Características, conceptos y conclusiones

Los modelos de Waclaw, Sottoriva y Sun exponen 3 formas distintas de diseñar y programar un modelo espacial. A pesar de ser modelos distintos, tienen características comunes y siguen un patrón parecido. Aparte de estos 3 modelos también se han estudiado los modelos de Poleszczuk ([16]), Iwasaki and Innan([17]) o Uchi et al. ([18]). En los que se repiten las ideas y conceptos presentados en los 3 que hemos desarrollado.

Por tanto, se dedica esta sección a enumerar las características que se han encontrado en estos modelos, definir los conceptos utilizados y extraer algunas conclusiones. Un objetivo es razonar cuáles de las características de cada modelo se consideran que son útiles y que se quieren adoptar para el modelo que se va a implementar.

Así, aparte de los eventos de mutación, reproducción y muerte propios de los modelos adimensionales, para modelos espaciales se pueden definir nuevos eventos, conceptos o factores:

- Deme: tal y como aparece en los modelos de Sottoriva y Sun, un deme es un conjunto de células que ocupa una posición fija en el espacio. Podría definirse más detalladamente como 'zona espacial definida por unas coordenadas (x,y,z en caso de un espacio tridimensional) que puede albergar una cantidad limitada de células'. Incluso en Waclaw podemos pensar que tenemos demes de tamaño 1, es decir, con capacidad para albergar una sola célula. El deme es utilizado para reducir los costes computacionales derivados de guardar la ubicación y simular el movimiento de cada célula por separado.
- Migración a territorios adyacentes: encontrado en los tres modelos descritos es el fenómeno por el cual las células (o demes) ocupan posiciones adyacentes en el espacio, normalmente al reproducirse.
- Migración a territorios más alejados: este fenómeno solo lo hemos visto en Waclaw; es el fenómeno por el cual las células (o demes) migran hasta posiciones no adyacentes en el espacio.

- Pushing: este concepto lo hemos encontrado en el modelo de Waclaw y también figura en el de Sottoriva: Es el resultado de que las células, al reproducirse, empujen a otras células ocupando su espacio.
- Factores microambientales: esto aparece en el modelo de Sottoriva, y responde a la necesidad de definir un nuevo factor en la evolución del tumor: el ambiente que rodea a las células.

Además, en general estos algoritmos tienen dos fases diferenciadas, en las que también puede modelarse una competencia:

1. Fase intrademe: propia de aquellos modelos donde hay demes, es la fase en la que se desarrolla y evolucionan las células del interior del deme. La **competencia intrademe** sería por tanto la competencia entre los genotipos por sobrevivir y dominar el interior de un deme.
2. Fase interdeme: esta es la fase más diferencial del modelo espacial, es el momento en el cual los demes, o las células que componen el deme, se mueven por el espacio ya sea a través de la reproducción o la migración. La **competencia interdeme** sería por tanto la competencia entre los genotipos por expandirse entre los demes y colonizarlos.

A partir de estas piezas cada científico construye su modelo. En base a distintos estudios o necesidades cada uno utiliza más unos factores que otros. Se han visto modelos en los que aparte de estos puntos se valoran algunos otros o se hacen consideraciones especiales, que suponen simplificaciones importantes: Sun, con el apoyo también de Waclaw, decide limitar el crecimiento del tumor sólo en los bordes del mismo. O Sottoriva piensa en un deme como en un conjunto de células con el mismo genotipo eliminando por tanto la competencia intrademe. Un resumen de las características principales se muestra en el siguiente cuadro 4.1, basado en la tabla de Robert Noble en su presentación no publicada [19]:

Modelo	Celulas por deme	Competencia interdeme	Competencia intrademe	Migración contigua	Migración no contigua	Pushing
Waclaw	1	Si	No	Si	Si	Si
Sottoriva	10000	Si	No	Si	No	Si
Sun	1000 to infinite	No	Si	Si	No	No

Cuadro 2.1: Características principales de los modelos espaciales discutidos; modificado de una tabla de Robert Noble en [19]

2.3. Paquete OncoSimulR

El modelo que se implementará estará integrado en el paquete *OncoSimulR* [20] (<https://github.com/rdiaz02/OncoSimul>). Este paquete ha sido desarrollado por Ramón Díaz Uriarte, Profesor Titular del Dpto. de Bioquímica de la Universidad Autónoma de Madrid. Pertenecer a Bioconductor, que es un proyecto de código abierto y que tiene el objetivo de desarrollar e integrar software para el análisis estadístico de datos de laboratorio en biología molecular. Este proyecto está escrito en el lenguaje de programación R, un lenguaje muy utilizado para el análisis estadístico y representación de datos.

OncoSimulR contiene código en R y en C++ relacionado con la simulación de modelos matemáticos para la investigación del cáncer. Al código en C++ se le llama desde R mediante

la librería **Rcpp**. Se utiliza C++ en el núcleo del algoritmo debido a que es mucho más rápido que R.

Una de las grandes funcionalidades que ofrece este paquete, la cual se utilizará en nuestro modelo, es la gran flexibilidad a la hora de indicar cómo se calcula el fitness de un individuo a partir de su genotipo. Este es un aspecto en el que no se ha profundizado demasiado hasta ahora, y es el momento de hacerlo. El inicio del problema es: ¿qué se quiere diseñar? Se quiere diseñar un modelo que permita simular la evolución de un conjunto de células a lo largo del tiempo. Y ¿de qué factores depende ese crecimiento? Depende del fitness relativo de cada célula o tipo de célula. Las células con mayor fitness serán las que dominen en el conjunto de células y se terminen expandiendo. Y el fitness depende a su vez del genotipo de la célula.

Por tanto, cuánto más flexible y compleja se consiga hacer la función que calcula el fitness de una célula, mayor cantidad de situaciones diferentes y de mayor complejidad se podrán simular. Por ello, se va a dedicar el resto de la sección en estudiar brevemente las diferentes opciones de especificación de fitness que aporta el paquete *OncoSimulR*.

Para ello se utiliza la siguiente función en R:

```
allFitnessEffects <- function(rT = NULL,
                             epistasis = NULL,
                             orderEffects = NULL,
                             noIntGenes = NULL,
                             geneToModule = NULL,
                             drvNames = NULL,
                             keepInput = TRUE)
```

La salida de esta función se le pasa a C++ en forma de una estructura. En esta estructura se almacenan todos los efectos del fitness que aporta cada gen. Los parámetros **geneToModule**, **drvNames**, y **keepInput** no interesan en este momento ¹, por lo que se van a explicar los otros cuatro:

1) noIntGenes: El nombre viene de 'genes que no interactúan', este parámetro es una lista de genes, cada cual lleva asociados su aportación al fitness si está mutados. Se muestra un ejemplo para aclararlo:

Indicamos los efectos del fitness con la función **allFitnessEffects** y nos ayudamos de la función **evalAllGenotypes** para evaluar los fitness de todos los genotipos posibles. El fitness de cada genotipo es el fitness multiplicativo $\prod(1 + fv)$ donde fv es el efecto de ese gen al fitness.

```
fe <- allFitnessEffects(noIntGenes = c("A" = -0.03, "B" = 0.05, "C" = 0.08))

evalAllGenotypes(fe, order = FALSE, addwt = TRUE)
## Genotype Fitness
##1      WT 1.00000
##2       A 0.97000
##3       B 1.05000
##4       C 1.08000
##5    A, B 1.01850
##6    A, C 1.04760
##7    B, C 1.13400
##8  A, B, C 1.09998
```

2) epistasis: Sirve para indicar los efectos del fitness en los que el efecto de la mutación de un gen sobre el fitness depende del estado (mutado o no mutado) de otros genes; la epistasia modela entonces desviaciones de un modelo en que el efecto de la mutación de cada gen en el fitness es independiente de los demás genes. Este parámetro es una lista de expresiones, cada cual lleva asociada su aportación al fitness. Se muestra un ejemplo para aclararlo:

¹Al tratarse de parámetros con otra funcionalidad del paquete que no cubriremos aquí

```
fe <- allFitnessEffects(epistasis = c("A:-B" = 0.1, "B:-A" = 0.4, "A : B" = 0.2,
"C:-A:-B" = 0.5, "C:A"=-0.5, "C:B"=0.4))

evalAllGenotypes(fe, order = FALSE, addwt = TRUE)
## Genotype Fitness
##1      WT      1.00
##2       A      1.10
##3       B      1.40
##4       C      1.50
##5     A, B      1.20
##6     A, C      0.55
##7     B, C      1.96
##8   A, B, C      0.84
```

Por ejemplo, en el número 7, se cumplen $B : -A$ y $C : B$, por lo que queda: $1,4 \cdot 1,4 = 1,96$.

3) orderEffects: Sirve para indicar los efectos del fitness en los que importa el orden de mutación de los genes. Este parámetro es una lista de expresiones, cada cual lleva asociada su aportación al fitness. Se muestra un ejemplo para aclararlo:

```
fe <- allFitnessEffects(orderEffects = c("A > B" = 0.4, "B > A" = 0.1,
"C > B > A" = 0.2, "A > C" = -0.3))

evalAllGenotypes(fe, order = TRUE, addwt = TRUE)
## Genotype Fitness
##1      WT      1.00
##2       A      1.00
##3       B      1.00
##4       C      1.00
##5     A > B      1.40
##6     A > C      0.70
##7     B > A      1.10
##8     B > C      1.00
##9     C > A      1.00
##10    C > B      1.00
##11  A > B > C      0.98
##12  A > C > B      0.98
##13  B > A > C      0.77
##14  B > C > A      1.10
##15  C > A > B      1.40
##16  C > B > A      1.32
```

Por ejemplo, en el número 11, se cumplen $A > B$ y $A > C$, por lo que queda: $1,4 \cdot 0,7 = 0,98$.

4) rT: Esta es una forma de especificar el fitness que está pensada para modelar escenarios en los que se especifican restricciones en el orden de acumulacion de mutaciones, dependencias que se especifican usando grafos. El efecto de cada gen depende entonces de que las restricciones de aparicion se hayan visto satisfechas o no. Cada nodo del grafo es un gen y tiene unos efectos de fitness asociados si se cumplen unas condiciones y otros si no se cumplen. Esto se indica mediante el parámetro `typeDep`:

- `typeDep = "MN"`: los efectos de fitness asociados a un nodo se cumplen si todos sus nodos padres están mutados, esto es equivalente a una relación de tipo AND.
- `typeDep = "SM"`: los efectos de fitness asociados a un nodo se cumplen si al menos un padre está mutado, esto es equivalente a una relación de tipo OR.
- `typeDep = "XMPN"`: los efectos de fitness asociados a un nodo se cumplen si sólo uno de los padres están mutados, esto es equivalente a una relación de tipo XOR.

Se muestran algunos ejemplos:

Mediante este código:

```
cs <- data.frame(parent = c(rep("Root", 2), "a", "b"),
  child = c("a", "b", "c", "c"),
  s = c(0.1, 0.2, 0.3, 0.3),
  sh = c(-0.8, -0.7, -0.5, -0.5),
  typeDep = "MN")

fe <- allFitnessEffects(cs)
```

Se genera el siguiente grafo al ejecutar `plot(fe)`:

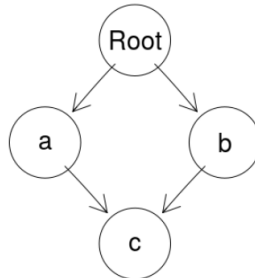


Figura 2.2: Grafo de ejemplo

El nodo *Root* siempre debe especificarse. Es un nodo abstracto, que sirve para enraizar todos los grafos, y sirve para representar un estado de 'nada mutado'.

Como ya se ha explicado antes, el parámetro `typeDep="MN"` indica que los efectos de fitness asociados a los nodos se tienen en cuenta sólo si el nodo de llegada tiene todos sus padres mutados, es decir, se tienen que cumplir todas las dependencias, como una relación AND.

El parámetro `s` indica cuales los efectos de fitness de los nodos si se cumplen las condiciones según lo indicado por el parámetro `typeDep` ya explicado. Mientras que el parámetro `sh` indica cuales los efectos de fitness si no se cumplen.

Una observación importante es que tanto `s` como `sh` deben ser consistentes con la forma de grafo. Es decir en este caso si se hubiera puesto `s = c(0.1, 0.2, 0.3, 0.4)`, hubiese dado error, ya que habría ambigüedad sobre que fitness coger cuando se cumplan las condiciones para el nodo `c`, si 0,3 o 0,4.

Se pintan las evaluaciones del fitness para explicarlas a continuación:

```
evalAllGenotypes(fe, order = FALSE, addwt = TRUE)
## Genotype Fitness
##1      WT      1.000
##2       a      1.100
##3       b      1.200
##4       c      0.500
##5    a, b      1.320
##6    a, c      0.550
##7    b, c      0.600
##8  a, b, c      1.716
```

- Los casos 2 y 3 son sencillos, un gen mutado en cada caso y se cumplen las dependencias ya que *Root* siempre está mutado.
- El caso 4 es también sencillo, está mutado el gen *c*, pero no están mutados *a* y *b*, por lo que se aplica `sh = -0.5`.
- En el caso 5 se encuentran mutados los genes *a* y *b* y cumplen las dependencias ya que *Root* siempre está mutado. El fitness total es $1,2 \cdot 1,1 = 1,32$.

- En el caso 6, para el gen *a* sí se cumplen las dependencias, pero para el gen *c* no, ya que *b* no está mutado. EL fitness asociado es $1,1 \cdot 0,5 = 0,55$. El caso 7 es análogo: $1,2 \cdot 0,5 = 0,6$.
- En el caso 8, se encuentran mutados todos los genes, se cumplen por tanto todas las condiciones y el fitness es $1,2 \cdot 1,1 \cdot 1,3 = 1,716$.

Se repite la operación con `typeDep="SM"`.

```
cs <-data.frame(parent = c(rep("Root", 2), "a", "b"),
               child = c("a", "b", "c", "c"),
               s = c(0.1, 0.2, 0.3, 0.3),
               sh = c(-0.8, -0.7, -0.5, -0.5),
               typeDep = "SM")

fe <- allFitnessEffects(cs)
evalAllGenotypes(fe, order = FALSE, addwt = TRUE)
## Genotype Fitness
##1      WT      1.000
##2       a      1.100
##3       b      1.200
##4       c      0.500
##5    a, b      1.320
##6    a, c      1.430
##7    b, c      1.560
##8  a, b, c      1.716
```

Y por último con `typeDep = "XMPN"`:

```
cs <-data.frame(parent = c(rep("Root", 2), "a", "b"),
               child = c("a", "b", "c", "c"),
               s = c(0.1, 0.2, 0.3, 0.3),
               sh = c(-0.8, -0.7, -0.5, -0.5),
               typeDep = "XMPN")

fe <- allFitnessEffects(cs)
evalAllGenotypes(fe, order = FALSE, addwt = TRUE)
## Genotype Fitness
##1      WT      1.00
##2       a      1.10
##3       b      1.20
##4       c      0.50
##5    a, b      1.32
##6    a, c      1.43
##7    b, c      1.56
##8  a, b, c      0.66
```

Aquí se observa la diferencia en el caso 8. Al no cumplirse las condiciones para *c* queda como fitness $1,1 \cdot 1,2 \cdot 0,5 = 0,66$.

Por supuesto, se pueden especificar los efectos del fitness mezclando todas estas formas, e incorporando especificaciones con grafos, con efectos de epistasis, con efectos de orden y/o con genes que no interactúan. Esto da gran flexibilidad y permite considerar situaciones tan complejas como se quiera.

2.3.1. Modelo en tiempo continuo de OncoSimulR: algoritmo de Mather

Hay varios algoritmos para la simulación de procesos estocásticos en tiempo continuo, tales como el *Gillespie's Direct Method* y el *Gillespie's First Reaction Method*. También existe una versión mejorada llamada *Next Reaction Method* (Gibson and Bruck [9]) algo más eficiente ya que aprovecha al máximo los datos tratando de actualizar solo los que sean necesarios.

El paquete OncoSimulR utiliza principalmente un modelo en tiempo continuo. Las simulaciones usan el algoritmo BNB de Mather et al (Mather et al., 2015 [10]) de simulación estocástica rápida para dinámica de poblaciones que explicamos a continuación. El algoritmo de Mather [10] es un ejemplo de algoritmo en tiempo continuo que se basa en todo un clásico: el algoritmo de Gillespie. Mather consigue una mejora sustancial de la velocidad de simulación con respecto a este modelo, gracias al algoritmo BNB (Binomial Negative-Binomial).

Se va a explicar el algoritmo básico *Gillespies's Direct Method* y posteriormente el implementado por Mather, que es el utilizado en el paquete OncoSimulR. El funcionamiento del método directo es el siguiente:

Se tiene una función $P(\mu, \tau)d\tau$ que nos da la probabilidad de que una reacción μ ocurra en tiempo τ . Si se integra $P(\text{reaction} = \mu)$ respecto de τ , se obtiene la probabilidad en el que se puede producir la reacción μ . Y si se integra $P(\tau)$ respecto de τ , se obtiene cuándo se va producir la siguiente reacción. El algoritmo quedaría:

1. Se inicializa $t \leftarrow 0$ y el número de moléculas de cada tipo.
2. Se elige μ y τ de acuerdo a lo dicho anteriormente. A esta fase también se la llama paso de Monte Carlo, se generan dos números aleatorios para determinar cuál es la siguiente reacción y en qué momento ocurrirá. La probabilidad de que una reacción dada sea elegida es proporcional al número de moléculas de los reactivos.
3. Se incrementa el tiempo y se actualizan las moléculas en consecuencia de la ejecución de la reacción μ .
4. Se vuelve al paso 2.

En este caso, y tratándose de un modelo que simula la reproducción, muerte y mutación de las células, se pueden considerar como reacciones precisamente los eventos celulares, esto es, la reproducción, la mutación y la muerte. Además, estos eventos dependen del fitness de las células, por lo que se podría separar el conjunto de células en grupos con el mismo genotipo. Así, se llamará N al número de conjuntos diferentes de genotipos. Por tanto, se tendrían $3N$ tipos diferentes de reacciones, y habría que calcular los tiempos de aparición de cada uno de ellos para ver cuál es el mínimo, y ejecutar esa reacción.

Pero para simplificar el problema, este modelo sugiere que como reacciones solo se van a considerar las mutaciones, por tanto, en cada iteración el avance de tiempo será el que transcurra entre una mutación y otra.

Ahora se explica con más detalle el algoritmo implementado en Mather et al ([10]), el BNB (Binomial, Negative-Binomial)::

1. Se inicializa el sistema con N clases de especies según el genotipo. Se especifica la población de cada especie con $n_i, i = 1, \dots, N$. Cada especie tendrá asignados sus propias tasas de nacimiento (g_i), muerte (λ_i) y mutación (μ_i).
2. Para cada clase se generarán N números r_i uniformemente distribuidos entre 0 y 1. Y para cada $i = 1, \dots, N$, se generará el tiempo t_i que quedaría hacia la siguiente mutación. Estos tiempos se calculan de acuerdo a unas ecuaciones matemáticas que se pueden ver con detalle en el anexo A de [10].
3. Se coge $t_m = \min(t_i)$ y se actualiza el tiempo $t = t + t_m$.
4. Se actualizarán el número de individuos de cada especie siguiendo el proceso que se describe más adelante.

5. Se selecciona la mutación que ocurre. Si esta genera un miembro que no pertenece a ninguna de las especies existentes, se crea una nueva especie con $n_{N+1} = 1$. En caso contrario se añade 1 a la correspondiente especie.
6. Las especies con 0 individuos se eliminan y se actualiza N al número actual de especies distintas.
7. Se vuelve al paso 2

El algoritmo de actualización del número de individuos es el siguiente: Sean g , λ y μ las tasas de nacimiento, muerte y mutación medias. Sea $p_M(t)$ la probabilidad de que ninguna mutación se haya producido, $p_E(t)$ es la probabilidad de extinción y $p_B(t) = \frac{gp_E(t)}{\lambda}$.

1. Para la especie mutada se genera un número aleatorio $\hat{m}$ de una binomial $B(n_0 - 1, 1 - (p_E(t_m)/p_M(t_m)))$. A continuación se actualiza $n_{i_m} = \hat{m} + 1 + NB(\hat{m} + 2, p_B(t))$ donde NB es una binomial negativa.
2. Para el resto de las especies se genera un numero aleatorio $\hat{m}$ de una binomial $B(n_0, 1 - (p_E(t_m)/p_M(t_m)))$. Si $\hat{m} = 0$ entonces se da a la especie por extinguida. En caso contrario se actualiza $n_i = \hat{m} + 1 + NB(\hat{m} + 2, p_B(t)) \forall i = 1, \dots, N$, donde NB es una binomial negativa.

Por tanto, este es un modelo en tiempo continuo que utiliza un algoritmo de crecimiento de tipo Branching Process Model. Es adimensional y los eventos celulares no dependen de factores ambientales.

3

Diseño y desarrollo

A lo largo de este capítulo se va a explicar cómo se ha llevado a cabo el diseño y desarrollo de un modelo de evolución tumoral espacial. Se explicará con detalle el modelo diseñado, tanto la parte matemática como la parte computacional, entendiendo sus parámetros de entrada y de salida, así como su funcionalidad.

3.1. Algoritmo

En primer lugar se va a explicar el algoritmo diseñado a grandes rasgos. Posteriormente se entrará en detalle explicando la razón de las decisiones tomadas. Por último veremos algunos aspectos concretos del algoritmo, los parámetros de entrada y un resumen de sus características. El objetivo era diseñar un modelo versátil, que permitiese simular la mayor cantidad de escenarios posibles y que fuera mínimamente restrictivo. El algoritmo ejecuta, en orden, cada uno de los siguientes cuatro subalgoritmos o fases.

1. Inicio: Se comienza con un deme de cierto tamaño y con unas mutaciones concretas y predefinidas por el usuario.
2. Fase intrademe: por cada deme se corre durante un tiempo específico el modelo en tiempo continuo (y adimensional) que viene implementado en el paquete OncoSimulR (con matices, pues como veremos más adelante el algoritmo original ha sido modificado para adaptarlo a este modelo).
3. Fase interdeme: por cada deme se eligen un número de células específicas que estocásticamente podrán migrar o no a territorios adyacentes. De igual manera por cada deme se elegirán otras células que podrán migrar a territorios más alejados.
4. Examen de las condiciones de parada: hasta aquí se produce una iteración; en esta fase se revisarán ciertas condiciones de parada y en caso de no cumplirse se volverá al punto 2 con la siguiente iteración.

3.1.1. Detalle de las cuatro fases del algoritmo

A continuación se van a explicar cada una de estas fases más detalladamente, indicando en cada una los parámetros de entrada con los que el usuario puede ajustar a sus necesidades la simulación.

1. **Inicio:** El inicio del algoritmo coincide con el inicio del tumor, por tanto implica la creación de un conjunto de células entre las cuales debe haber una o varias que tengan mutaciones en su genotipo. El conjunto de células creadas se llamará deme, y ocupará la posición (0,0,0) en un espacio tridimensional. En el resto de posiciones del espacio también habrá demes, todos ellos formados por células sin mutar. El usuario tendrá la capacidad de especificar el inicio del tumor con varios parámetros:
 - **initMutant:** es una cadena de strings que representa las mutaciones que tendrá en un comienzo el genotipo de las células que originan el tumor. El usuario puede por tanto elegir cuál es la mutación que empieza el tumor con el objetivo de estudiar qué mutaciones consiguen fijarse, mantenerse o expandirse con más facilidad, o cuáles mueren a los pocos momentos. Es un parámetro obligatorio.
 - **initSize:** representa el numero de células que tiene el deme con el que comenzamos la simulación.
 - **initMutantPercentage:** indica el porcentaje del número de células del primer deme (**initSize**) que están mutadas (según **initMutant**) al iniciar la simulación. Por defecto 0.1.
2. **Fase intrademe:** En esta fase se modela lo que ocurre dentro de cada deme. Por simplicidad y tiempo de ejecución, y de igual manera que en los modelos estudiados, solo se simulará el crecimiento del tumor en los demes que hayan tenido alguna vez una mutación a lo largo del proceso. Esto quiere decir que en la primera iteración solo se simulará el crecimiento del deme creado en la fase de inicio. Para simular el crecimiento del tumor en el interior del deme se utilizará un modelo en tiempo continuo basado en el existente en el paquete OncoSimulR, y que será explicado más adelante. El usuario podrá personalizar esta fase de la simulación con algunos parámetros heredados del modelo en tiempo continuo:
 - **mu:** Es la probabilidad de mutación de cada gen del genotipo. Por defecto es 1e-6, que es un valor que está fundamentado en evidencia empírica sobre la tasa de mutación de un gen de tamaño medio, y puede hacerse distinto entre distintos genes.
 - **sampleEvery:** Con qué frecuencia se toma la muestra de toda la población. Puede ser necesario tener un valor pequeño aquí para minimizar posibles problemas numéricos (como un gran aumento del tamaño de la población). El modelo llama a funciones internas que controlan el estado de la simulación y actualizan parámetros. Este parámetro es obligatorio, una buena inicialización para el modelo espacial sería 0.1.
 - **finalTime:** Es el tiempo durante el que se corre el modelo en tiempo continuo en cada iteración y en cada deme. Este parametro es obligatorio.
3. **Fase interdeme:** Esta es la fase que más caracteriza al modelo espacial. Se realiza la expansión de las células cancerígenas. En cada iteración se recorren de nuevo los demes que se han recorrido en la fase anterior y se escoge de cada uno una cantidad aleatoria de células que se expandirán a los espacios adyacentes y otra cantidad que podrán migrar a territorios más alejados.

La migración de las células supone la creación de un nuevo deme si este territorio estaba vacío, o la invasión de un deme si el territorio ya estaba ocupado. Al crearse un deme,

este se inicializa con `initSize` células sin mutar más las que lleguen durante la migración. Tras la finalización de la fase interdeme se completa una iteración. La personalización que puede hacer el usuario en esta fase se realiza con los siguientes parámetros.

- **migrationProb**: indica, por cada deme, la probabilidad de que haya migración a espacios adyacentes. Por defecto 0.5.
 - **largeDistanceMigrationProb**: indica, por cada deme, la probabilidad de que haya migración a espacios mas alejados (entre 10 y 35 espacios por cada eje de coordenadas). Por defecto 1e-6.
 - **maxMigrationPercentage**: es el máximo porcentaje de células de cada deme que migrarían. Cada vez que se realiza una migración (en cada iteración, en cada deme y ya sea a espacios adyacentes o alejados) se escogerán al azar un conjunto de células entre 1 (si la probabilidad de migración se ha cumplido) y este porcentaje por el tamaño del deme. Por defecto vale 0.2.
4. **Examen de condiciones de parada**: A pesar de haberse completado ya una iteración, el algoritmo esta diseñado para que las condiciones de parada se estudien tras la fase intrademe de la iteración siguiente. Esto tiene el objetivo de no parar la simulación justo después de la migración que deja territorios sin desarrollar que acaban de ser colonizados o demes en proceso de invasión. Así, el algoritmo siempre se inicia y se finaliza con una simulación de crecimiento intrademe. El usuario puede determinar dos condiciones de parada:
- **spatialDemesMax**: Indica el número máximo de demes que se pueden tener en una simulación. Por defecto -1.
 - **spatialIterMax**: Indica el número máximo de iteraciones que se pueden alcanzar en una simulación. Por defecto 20.

Es obligatorio definir uno de estos parámetros a un valor mayor que 0. Si una de estas condiciones se cumplen despues de la fase intrademe de la siguiente iteración, el algoritmo llega a su fin. Si no, se continua con la fase interdeme hasta la siguiente iteración.

3.1.2. Parámetros de entrada y de salida

Como se ha ido mostrando en la sección anterior la simulación puede ser personalizada por parte del usuario mediante varios parámetros. En esta sección mostramos el total de parámetros del que se dispone, explicando aquellos que no han salido en la sección anterior, ya que no pertenecen a una fase concreta del algoritmo. También mostramos la salida del algoritmo tras su ejecución.

- **fe**: son los efectos del fitness que se describían en el la seccion 2.3. Es un parámetro de entrada necesario para cualquier simulación. Es un parámetro obligatorio.
- **model**: hay varios entre los que elegir ("Bozic", ".^{Exp}", "McFarlandLog"), se utilizan sobre todo en la fase intrademe y afectan entre otras cosas a la variabilidad del tamaño de la población. Para el algoritmo espacial se debe elegir 'Constante', que ha sido creado a propósito para este caso. Se comentará este aspecto más adelante. Este parámetro es obligatorio.
- **cteSize**: va ligada a usar el `model = 'Constante'`. Indica el tamaño en el que se quiere que se establezca la población de un deme cuando sus células adquieren el fitness máximo. Por defecto será igual a `2·initSize`.

- **spatialModel**: TRUE o FALSE. Indica si se quiere hacer uso del algoritmo descrito anteriormente, es decir del modelo espacial. Por defecto es FALSE, por tanto se debe poner a TRUE para correr el modelo espacial.
- **spatialKeepEvery**: indica cada cuantas iteraciones se guarda el estado de simulación para su posterior visualización al final del algoritmo, se comentará con detalle más adelante. Por defecto vale 1.
- **spatialVerbosity**: a mayor valor más información saldrá por pantalla sobre el estado de la simulación. El valor por defecto y el recomendado es 2.
- **spatialDeleteDemesWithoutMutations**: vale TRUE si queremos eliminar al final de cada iteración aquellos demes en los que todas sus células no tengan mutaciones. Eliminar significa que estos demes no serán evaluados en las fases intrademe e interdeme de las iteraciones siguientes, hasta que vuelvan a recibir una célula migrada. Estudiaremos las diferencias de tener este parámetro a TRUE o FALSE más adelante. Por defecto será FALSE.

El resto de parámetros son los ya indicados en los apartados anteriores: `mu`, `sampleEvery`, `initSize`, `finalTime`, `initMutant`, `initMutantPercentage`, `migrationProb`, `largeDistanceMigrationProb`, `maxMigrationPercentage`, `spatialDemesMax`, `spatialIterMax`.

Al ejecutar el algoritmo este nos devuelve unos valores, estos son:

- **SpatialSummary**: resumen del resultado de la simulación. Es un dataframe con las siguientes columnas: **Genotypes** (lista de genotipos), **Population** (población de células de cada genotipo), **Demes** (demes en los que aparece cada genotipo), **WinDemes** (demes en los que cada genotipo es el genotipo mayoritario de ese deme).
- **TotalPopulation**: número total de células al final de la simulación.
- **NumIter**: número de iteraciones alcanzadas al final de la simulación.
- **NumDemes**: número de demes final.
- **TimeForIter**: tiempo que transcurre para el tumor en cada iteración. Es el valor del parámetro `finalTime`.
- **Simulation**: es una lista con toda la información de genotipos, demes y población guardada cada `spatialKeepEvery` iteraciones. Este parámetro nos ofrece información de cómo ha ido la evolución del tumor a lo largo del tiempo, además, como veremos más adelante, con este parámetro se podrá ver gráficamente la simulación.

Más adelante se verán ejemplos de ejecución del modelo y los resultados originados.

3.1.3. Otros aspectos: estructura de vecindario, tamaño del deme y simulación gráfica

A lo largo de la implementación de este algoritmo se han ido tomando decisiones buscando optimizar el coste computacional del software y su funcionalidad. Además se pueden elegir diferentes caminos en otros aspectos y algunos de ellos se han dejado a elección del usuario mediante parámetros de entrada. En esta sección se describen algunas decisiones o aspectos del algoritmo que son importantes y que no se han comentado anteriormente.

Estructuras de vecindario:

Una de las elecciones que había que hacer era cómo definir las distancias o el espacio tridimensional. En general se vislumbran 3 opciones:

- Von Neumann: una simplificación del modelo de Moore en el que cada deme tiene 6 vecinos.
- Hexagonal: esta es la opción más novedosa, la estructura hexagonal es apoyado por algunos modelos como el de Iwasaki ([17]).
- Moore: el escogido y el más habitual, cada deme tiene 26 vecinos adyacentes.



Figura 3.1: Estructuras espaciales de vecinos (Iwasaki [17]).

Se planteó la posibilidad de proveer al usuario la capacidad de elegir la estructura espacial que prefiera. Finalmente no se implementó, así que queda como un aspecto a mejorar en futuras versiones.

Tamaño del deme:

Este ha sido uno de los problemas más controvertidos. El debate era ponerle una cota al tamaño o dejar crecer los demes libremente. Para solucionar el problema se recurre a la definición que se ha dado al deme anteriormente (apartado 2.3): 'Conjunto de células que ocupan una posición fija en el espacio' o 'zona espacial definida por unas coordenadas que puede albergar una cantidad limitada de células'. Según esta definición el tamaño del deme no podía ser infinito, ya que en una zona espacial acotada no pueden caber infinitas células. Sin embargo esto podía romper con otra base de la genética selectiva y los tumores: si se producía en algunas células del deme una mutación conductora que proveía a estas células de una tasa de crecimiento y supervivencia enormes, entonces se debería permitir que este genotipo se expandiera, quizás no infinitamente, pero sí en gran medida.

La solución que se tomó fue poner una cota de estabilización, que indica el valor alrededor del cual va a oscilar el tamaño del deme cuando todas sus células tienen el fitness máximo, este valor será el indicado por el parámetro `cteSize`. Si no todas las células tienen el fitness máximo la cota será proporcionalmente menor. Por ejemplo, si una célula tiene 3 posibles genotipos, uno que le confiere fitness (a) 1, otro (b) 1.5 y otro (c) 2, y se define `initSize=1000` y `cteSize=2000`, entonces si el deme tiene en un momento dado 800 células con genotipo b (y ninguna célula más), su población podrá crecer hasta 1500 células. Así, si se produce una mutación que produce una ventaja en el fitness, lo que va a ocurrir es que durante la fase intrademe esta mutación puede ocupar una mayor parte de ese deme. Posteriormente, durante la fase interdeme, mediante la migración, la mutación se podrá expandir a otras áreas espaciales, aumentando su dominio rápidamente por otros demes. Si se define `initSize=cteSize` este efecto se anulará pues el tamaño de estabilización del deme será el mismo independientemente del fitness medio de sus células.

Algo que deriva de esto y que merece la pena comentar es el hecho de que al iniciarse la simulación se asume que estamos en un espacio tridimensional ocupado por demes con `initSize` células

en su interior, y que se tratan sólo aquellos que hayan contenido mutaciones. Quizás también puede ser interesante iniciar el espacio con todos los demes vacíos, sin ninguna célula, representando la evolución de un tumor en espacios vacíos. Este escenario es el planteado por Waclaw y Sottoriva ([12] [13]) y este modelo permite simular una situación parecida (en el futuro sería interesante permitir que los demes estuvieran completamente vacíos, pero de momento no se puede). Al elegir un valor `initSize` « `cteSize` los demes inicialmente se encontrarán relativamente vacíos en comparación con el tamaño que alcanzarán si se produce una mutación ventajosa en su interior. En este contexto cobra importancia el parámetro `spatialDeleteDemesWithoutMutations` que si esta a `TRUE` permite dejar de evaluar aquellos demes que tienen todas sus células sin mutar, quitando carga computacional al modelo. Se verá más adelante las consecuencias que puede tener poner este parámetro a `TRUE` o a `FALSE`.

Simulación gráfica

Uno de los puntos que se consideraban importantes en este trabajo era poder visualizar gráficamente la evolución del tumor. Para ello se utiliza el parámetro `spatialKeepEvery`, la información de salida localizada en la variable `Simulation` y una función en R que se ha implementado con este fin. Cada `spatialKeepEvery` iteraciones se guarda el estado actual de la simulación, esto es: por cada deme su posición y genotipo mayoritario de cada uno, y en general por cada genotipo se guarda el número de células que lo tienen, el número de demes en los que está presente y el número de demes en los que ese genotipo es el mayoritario. Si se escoge `spatialKeepEvery` ≤ 0 solo se guardará el estado final de la simulación.

Toda esta información se guarda en el objeto de salida `Simulation`, de manera que el usuario, al terminar la ejecución, podrá llamar a la función `simulation3D` desde R y representar gráficamente la evolución del tumor. Se verán ejemplos más adelante.

3.1.4. Modificación del algoritmo y modelo de OncoSimulR para usar demes de población aproximadamente constante

EL algoritmo en tiempo continuo utilizado por el modelo espacial emplea el algoritmo de Mather, pero se han introducido modificaciones para permitir que la población oscile en torno a un número de células preespecificadas. Este algoritmo es el que se denomina 'Constante' (i.e., en OncoSimulR, donde antes se podían especificar 'Exp' y 'McFL' ahora podemos además especificar 'Constante')

Simplificando, en el modelo en tiempo continuo se parte de la fórmula básica de crecimiento poblacional: $n(t) = n(0) \cdot e^{(b-d)t}$ ('b' de nacimiento y 'd' de muerte) donde se modifican los parámetros b y d según las mutaciones sean beneficiosas o perjudiciales. Para la modificación que se va realizar se considera un conjunto de especies o clones $i \in I$ con poblaciones $n_i(t)$ en tiempo t y con tasas de crecimiento $g_i = b_i - d_i$. La idea es añadir a esta tasa un valor constante r de manera que la tasa de crecimiento quede como $g_i - r$. Así, la población en tiempo $t + \Delta t$ sería:

$$n_i(t + \Delta t) = n_i(t) \cdot e^{(g_i - r)\Delta t}$$

Y la población total quedaría como:

$$N(t + \Delta t) = \sum_{i \in I} n_i(t) \cdot e^{(g_i - r)\Delta t}$$

Se quiere que $N(t + \Delta t) = CTE$. Donde `initSize` < `CTE` < `cteSize`. Por tanto, usando Taylor queda:

$$CTE = N(t + \Delta t) = \sum_{i \in I} n_i(t) \cdot e^{(g_i - r)\Delta t} = \sum_{i \in I} n_i(t) \cdot (1 + (g_i - r)\Delta t) =$$

$$= \sum_{i \in I} n_i(t) + \sum_{i \in I} n_i(t) \cdot ((g_i - r)) \Delta t$$

Se divide todo por $\sum_{i \in I} n_i(t) = N(t)$:

$$\frac{CTE}{N(t)} = 1 + \left(\frac{\sum_{i \in I} n_i(t) g_i}{N(t)} - r \right) \Delta t$$

Y se despeja r:

$$\begin{aligned} r \Delta t &= 1 + \frac{\sum_{i \in I} n_i(t) g_i \Delta t}{N(t)} - \frac{CTE}{N(t)} \Rightarrow r = \frac{1}{\Delta t} + \frac{\sum_{i \in I} n_i(t) g_i}{N(t)} - \frac{CTE}{N(t) \Delta t} \Rightarrow \\ &\Rightarrow r = \frac{\sum_{i \in I} n_i(t) g_i}{N(t)} + \frac{N(t) - CTE}{N(t) \Delta t} \end{aligned}$$

Por tanto la modificación que se ha realizado consiste en cambiar la tasa de muerte de las células sumando ese valor de r . De esa forma el valor total de la población oscilará en torno al valor diana, CTE. La oscilación será menor cuanto más frecuentemente actualicemos toda la población y los parámetros de nacimiento y muerte, lo que está controlado por el parámetro `sampleEvery`. Disminuir en exceso `sampleEvery` tiene el inconveniente de incrementar el coste computacional del algoritmo ([10])

3.1.5. Resumen de características

El algoritmo desarrollado cumple con los requisitos para ser un modelo espacial. Cuenta con un **espacio tridimensional** en el que se hayan los **demes**, utilizados para disminuir la carga computacional que supone guardar y simular el movimiento de millones de células por separado. Dentro de cada deme se simula el modelo en tiempo continuo, donde las células compiten por mantener su supervivencia o expandirse dentro de las dimensiones del mismo; por tanto existe **competencia intrademe**. Además, cada deme emite **migrantes** hacia otras posiciones del espacio, tanto cercanas como lejanas, de manera que las células que hayan conseguido ser más mayoritarias en el deme tendrán más posibilidades de migrar y conquistar otros demes; por tanto existe **competencia interdeme**.

El algoritmo desarrollado no implementa el **pushing**, es decir, en ningún momento una célula va a abrirse hueco empujando a otra. No se vio necesario ya que añadía una carga computacional enorme para un beneficio cuestionable; las células que migran a otros demes entran sin limitación y a continuación, en la fase intrademe, será el algoritmo en tiempo continuo el que decida qué células mantienen su sitio en ese deme y cuáles mueren en el intento. Tampoco se introducen **factores microambientales**, es decir, las células solo interactúan con otras células y no hay ningún factor ajeno y cambiante según la posición de las células que afecte al desarrollo del tumor. Estos factores pueden ser un buen punto a investigar en el futuro, aunque la mayor parte de los simuladores de crecimiento tumoral que se centran en aspectos genéticos tampoco los tienen en cuenta.

4

Instalación, ejecución, pruebas y resultados

En este capítulo, se va a contar brevemente como se realiza la integración del mismo en el paquete OncoSimulR y cómo se puede acceder al código para su instalación. Posteriormente se mostrarán algunas ejecuciones del algoritmo, mostrando la importancia de algunos parámetros. También se aplicará el modelo a un escenario clásico, mostrando la visualización gráfica implementada y analizando los resultados obtenidos. Se finalizará el capítulo mostrando el rendimiento en tiempo y memoria del algoritmo.

4.1. Instalación

El algoritmo diseñado es implementado en R y C++ e introducido dentro del paquete OncoSimulR. Para integrar código de C++ en R se debe usar la librería Rcpp, que permite llamar a C++ desde R, y a R desde C++, e intercambiar objetos de R entre los dos lenguajes. La ejecución del modelo se realiza desde R, llamando a una función que internamente utiliza C++ debido a que es mucho mas eficiente computacionalmente que R. Sin embargo, los resultados y la parte gráfica se muestran con R.

El código que se puede instalar se encuentra en el repositorio https://bitbucket.org/aparramon/tfm_2/src/master/. Es un código listo para usarse y esta a falta de introducirlo en el repositorio oficial de OncoSimulR (<https://github.com/rdiaz02/OncoSimul>). Para instalarlo basta con seguir los siguientes pasos (a partir del paso 3 si no se usa ubuntu):

1. Tener la versión de R 3.3.0 en adelante . Ubuntu, por defecto, no la tiene. Pero debería ser sencillo ponerla, puede ser de ayuda consultar el siguiente enlace: <https://cran.r-project.org/bin/linux/ubuntu/README.html>

2. Instalar algunos paquetes conflictivos:

```
apt-get install r-cran-igraph  
apt-get install r-cran-car
```

3. Instalar Oncosimul desde Bioconductor, ya que se arrastran las dependencias (mas información en <http://bioconductor.org/packages/3.3/bioc/html/OncoSimulR.html>):

```
source("https://bioconductor.org/biocLite.R")
biocLite("OncoSimulR")
```

4. Descargar el proyecto OncoSimulR desde https://bitbucket.org/aparramon/tfm_2/src/master/
5. Desde una terminal entrar en el directorio donde se encuentra el paquete descargado (debe estar descomprimido) y ejecutar:

```
R CMD INSTALL OncoSimulR
```

Es importante indicar que cuando el código este en el repositorio oficial de OncoSimulR bastará con ejecutar únicamente el paso 3.

4.2. Ejecución y resultados

Una vez instalado, para ejecutar el algoritmo simplemente hay que definir unos efectos de fitness y llamar a la función `oncoSimulIndiv()` con los argumentos correctos. Una ejecución sencilla en la que el algoritmo cogerá la mayoría de los valores por defecto sería:

```
# Se importa la librería OncoSimulR
library(OncoSimulR)

# Se definen los efectos de fitness
fe <- allFitnessEffects(epistasis = c("A:-B:-C"=0.05, "-A:B:-C"=0.05, "-A:-B:C"=0.05,
                                     "A:B:-C"=0.045, "A:-B:C"=0.045, "-A:B:C"=0.045,
                                     "A:B:C"=0.055))

# Se simula el modelo espacial
ep2 <- oncoSimulIndiv(fe,
  model = 'Constante',
  sampleEvery = 0.5,
  finalTime = 500,
  initMutant = c("A"),
  spatialModel = TRUE)
```

La ejecución de este código provoca la siguiente salida por pantalla. Las últimas líneas donde se hace un resumen de la simulación son variables de una simulación a otra, salvo que fijemos la semilla previamente con `set.seed(100)` por ejemplo:

```
Iteracion 1 (2 demes)
Iteracion 2 (2 demes)
Iteracion 3 (3 demes)
Iteracion 4 (4 demes)
Iteracion 5 (7 demes)
Iteracion 6 (9 demes)
Iteracion 7 (13 demes)
Iteracion 8 (20 demes)
Iteracion 9 (26 demes)
Iteracion 10 (38 demes)
Iteracion 11 (52 demes)
Iteracion 12 (67 demes)
Iteracion 13 (93 demes)
Iteracion 14 (120 demes)
```

```
Iteracion 15 (151 demes)
Iteracion 16 (199 demes)
Iteracion 17 (253 demes)
Iteracion 18 (316 demes)
Iteracion 19 (394 demes)
Iteracion 20 (477 demes)
FINAL: Numero máximo de iteraciones alcanzado: 20

Final Population: 455427 (477 demes 20 iterations)
Genotype: A --- Population: 454525 --- Demes: 477 --- WinDemes: 476
Genotype: A, B --- Population: 44 --- Demes: 4 --- WinDemes: 0
Genotype: A, C --- Population: 858 --- Demes: 3 --- WinDemes: 1
```

Por cada genotipo se indica la cantidad de células que tienen ese genotipo (Population), en la cantidad de demes en los que aparecen (Demes) y en la cantidad de demes en los que es el genotipo mayoritario (WinDemes). Se puede acceder a los parámetros de salida mediante la consola de R, por ejemplo:

```
> ep2$TotalPopulation
[1] 455427
> ep2$NumIter
[1] 20
> ep2$NumDemes
[1] 477
> ep2$TimeForIter
[1] 500
> ep2$geneNames
[1] "A" "B" "C"
```

A continuación se van a contrastar las salidas cambiando el valor de algunos parámetros, con el objetivo de ver cómo influyen en la simulación del algoritmo. Para realizar estas pruebas se va a utilizar un genotipo inventado formado por los genes 'A', 'B' y 'C' y que tiene los siguientes efectos de fitness:

```
> evalAllGenotypes(fe, order = FALSE)
Genotype Fitness
1      A      1.050
2      B      1.050
3      C      1.050
4    A, B      1.045
5    A, C      1.045
6    B, C      1.045
7  A, B, C      1.055
```

Como se ve son todos los fitness parecidos, pero para llegar al fitness más ventajoso hay que pasar por un fitness algo perjudicial.

Pruebas con relación de parametros `initSize` y `cteSize`

Lo que se quiere mostrar en este contraste es que el tumor se expande más rápido si `initSize` es mucho menor que `cteSize`, o dicho de otra manera, si tiene que competir con menor cantidad de células o se expande por territorios más vacíos. Si `cteSize` es mucho mayor que `initSize`, en la migración la cantidad de células que llegan a un nuevo deme representará un porcentaje mayor con respecto a las iniciales, favoreciendo por tanto la expansión del tumor.

Se intenta aislar este parámetro para ver su importancia. Para ello se fija $\mu=1e-10$ (valor muy bajo), `migrationProb=1` y `largeDistanceMigrationProb=0`. Se empieza probando con `initSize=300` y `cteSize=300`. Al haber muy pocas mutaciones hay que asegurarse de que las células tumorales ganan en el deme inicial y por ello se pone un valor de `initMutantPercentage` alto

```
ep2 <- oncoSimulIndiv(fe,
  model = 'Constante',
  sampleEvery = 0.5,
  initSize = 300,
  cteSize = 300,
  initMutantPercentage = 0.7,
  maxMigrationPercentage = 0.2,
  migrationProb = 1,
  largeDistanceMigrationProb = 0,
  mu = 1e-10,
  finalTime = 100,
  initMutant = c("A"),
  spatialModel = TRUE,
  spatialVerbosity = 2,
  spatialIterMax = 15,
  spatialDeleteDemesWithoutMutations = TRUE)
```

Tras 10 ejecuciones los resultados varían en torno a los siguientes valores medios:

```
Final Population: 379578 (1264 demes 15 iterations)
Genotype: A --- Population: 260260 --- Demes: 886 --- WinDemes: 878
Genotype: --- Population: 119318 --- Demes: 470 --- WinDemes: 386
```

Aproximadamente una tercera parte de los demes tienen células sin mutar y en un 25 % de ellos son las células mayoritarias. Se prueba ahora con `initSize=100`. Tras 10 ejecuciones los resultados varían en torno a los siguientes valores medios:

```
Final Population: 321719 (1173 demes 15 iterations)
Genotype: A --- Population: 321584 --- Demes: 1172 --- WinDemes: 1172
Genotype: --- Population: 135 --- Demes: 7 --- WinDemes: 1
```

Se observa que al expandirse por un territorio casi vacío el tumor tiene menos dificultades en conquistarlo.

Pruebas con parámetro `spatialDeleteDemesWithoutMutations`

Se va a mostrar la influencia que tiene este parámetro. Para aislarlo de la influencia del resto de parámetros se fijan los demás como antes, esta vez se utiliza `initSize=100` y `cteSize=100`.

```
ep3 <- oncoSimulIndiv(fe,
  model = 'Constante',
  sampleEvery = 0.5,
  initSize = 100,
  cteSize = 100,
  initMutantPercentage = 0.7,
  maxMigrationPercentage = 0.2,
  migrationProb = 1,
  largeDistanceMigrationProb = 0,
  mu = 1e-10,
  finalTime = 100,
  initMutant = c("A"),
  spatialModel = TRUE,
  spatialVerbosity = 2,
  spatialIterMax = 15,
  spatialDeleteDemesWithoutMutations = FALSE)
```

Se toman tiempos con `proc.time()`. Tras 10 ejecuciones los resultados varían en torno a los siguientes valores medios:

```
Final Population: 126300 (1254 demes 15 iterations)
Genotype: A --- Population: 18054 --- Demes: 179 --- WinDemes: 179
Genotype: --- Population: 108246 --- Demes: 1080 --- WinDemes: 1075
```

```
user  system elapsed
3.700   0.004   3.699
```

Lo que se observa es que la cantidad de demes con genotipo mayoritario 'A' varía entre 52 (mínimo obtenido) y 243 (máximo obtenido), y los tiempos de ejecución están sobre los 3.5 segundos. Fijar el parámetro `spatialDeleteDemesWithoutMutations` debería mantener esta cantidad de demes en torno a esos valores y disminuir enormemente el tiempo de ejecución. Los resultados medios tras varias ejecuciones son:

```
Final Population: 14917 (152 demes 15 iterations)
Genotype: A --- Population: 14724 --- Demes: 152 --- WinDemes: 150
Genotype: --- Population: 193 --- Demes: 4 --- WinDemes: 2
```

```
user  system elapsed
0.764   0.004   0.768
```

El tiempo de ejecución se ha disminuido a una quinta parte, y los resultados son muy parecidos, tras 10 ejecuciones el valor mínimo de demes con genotipo 'A' ha sido 74 y el máximo 190. El uso de este parámetro representa una utilidad en este caso al tener el parámetro `mu` (tasa de mutación) muy bajo por lo que aumentar la cantidad de demes no aumenta apenas la probabilidad de que aparezcan nuevas mutaciones, pero debe usarse con precaución, se verá a continuación.

Caso extremo con demes de poco tamaño: `cteSize` ≤ 10

Al margen de la relación vista anteriormente de `cteSize` con `initSize` es importante recalcar que valores muy bajos de este parámetro (<5) pueden provocar la muerte instantánea de muchos demes por quedarse sin células. En la teoría es posible simular un modelo espacial en el que cada posición en el espacio es ocupada por una célula (como en el modelo de Waclaw [12]) con este algoritmo, pero en la práctica estos demes se extinguirán con suma facilidad.

Pero hay otro efecto también muy importante. En demes en las que se tienen 2 células conductoras y 8 neutras, la probabilidad de que las 2 mutadas sobrevivan es mucho menor que en una situación de 80-20, sin embargo entre la situación de 80-20 y la de 800-200 no hay apenas diferencia. Esto es pura genética evolutiva, durante el modelo en tiempo continuo, con el paso del tiempo las células se reproducen o mueren, el azar hace que sea más probable que mueran esas 2 a que mueran 20 en una situación de 80-20. Con tamaños de población pequeños, los efectos de la deriva genética son muy importantes, este es un fenómeno bien conocido en biología evolutiva [21]. También dependen bastante los efectos de fitness, para estos ejemplos se han definido unas ventajas de fitness pequeñas (1.05 sobre 1); si se aumenta a 1.5 sobre 1 también aumentan las posibilidades de supervivencia.

Además, que se produzca una mutación en un deme de 10 células es mucho menos probable a que se produzca en un deme de 1000 células. Normalmente la variabilidad genética total de los demes de 10 células es mínimo, albergando a la vez un genotipo o dos como mucho.

La forma de intentar simular demes con muy poca cantidad de células es bajando mucho el parámetro `finalTime`. Este parámetro es importante que este en relación con `cteSize`, y si se aumenta uno se debería aumentar el otro. Esto es razonable, pues si se mantiene el mismo `finalTime` (se recuerda que este parámetro mide el tiempo durante el que se corre la fase intrademe) al pasar de demes de 1000 células a 10 células estaríamos disminuyendo la relación entre fase intrademe y fase interdeme, y la parte de migración perdería importancia; no tiene sentido simular durante grandes cantidades de tiempo la evolución en el interior de un deme en el que solo hay 10 células, sin dejar a estas células interactuar con las de otros demes. Por otro lado los demes se introducen para reducir la carga computacional, pero si se utilizan demes de tamaño bajo, no se consigue este ahorro.

Se hace una prueba, se fija `initSize=10`, `cteSize=10`. Además se coloca `initMutantPercetaje` alto para evitar que las mutaciones mueran en la primera mutación, `mu` se establece a $1e-5$. Se aumenta también el fitness de las células:

Genotype	Fitness
1 A	1.50
2 B	1.50
3 C	1.50
4 A, B	1.45
5 A, C	1.45
6 B, C	1.45
7 A, B, C	1.55

Los resultados obtenidos varían en torno al siguiente:

```
Final Population: 77349 (7540 demes 25 iterations)
Genotype: A --- Population: 31462 --- Demes: 2966 --- WinDemes: 2939
Genotype: --- Population: 44149 --- Demes: 4482 --- WinDemes: 4435
Genotype: A, B --- Population: 0 --- Demes: 0 --- WinDemes: 0
Genotype: B --- Population: 838 --- Demes: 79 --- WinDemes: 79
Genotype: C --- Population: 797 --- Demes: 80 --- WinDemes: 77
Genotype: A, C --- Population: 103 --- Demes: 11 --- WinDemes: 10
```

Lo más significativo es la gran cantidad de demes que aparecen sin mutaciones. Si se prueba con `initSize=5`, `cteSize=10` los resultados muestran más células con genotipo 'A':

```
Final Population: 33666 (3785 demes 25 iterations)
Genotype: A --- Population: 29072 --- Demes: 2869 --- WinDemes: 2866
Genotype: --- Population: 4230 --- Demes: 895 --- WinDemes: 885
Genotype: A, C --- Population: 295 --- Demes: 26 --- WinDemes: 26
Genotype: A, B --- Population: 28 --- Demes: 3 --- WinDemes: 3
Genotype: B --- Population: 41 --- Demes: 5 --- WinDemes: 5
```

Sin embargo hay muchos menos demes. Esto es debido al hecho de que demes con solo 5 células pueden extinguirse con facilidad y hay que tener en cuenta que se ha puesto el parámetro de `migrationProb` a 0.5, lo que implica que de las 5 células pueden migrar hasta 3, dejando a algunos demes con 2 células al principio de la siguiente iteración. Se prueba con `spatialDeleteDemesWithoutMutations=TRUE` y las diferencias no son demasiado grandes, lo que apunta a que en este caso el uso de este parámetro puede ser de utilidad para reducir la carga computacional:


```

Final Population: 23203 (2122 demes 25 iterations)
Genotype: A --- Population: 22694 --- Demes: 2094 --- WinDemes: 2082
Genotype: --- Population: 140 --- Demes: 36 --- WinDemes: 10
Genotype: A, B --- Population: 54 --- Demes: 6 --- WinDemes: 6
Genotype: A, C --- Population: 283 --- Demes: 22 --- WinDemes: 21
Genotype: C --- Population: 0 --- Demes: 0 --- WinDemes: 0
Genotype: B --- Population: 32 --- Demes: 3 --- WinDemes: 3

```

Las pruebas con tamaños de deme muy pequeños no dejan unos resultados muy convincentes, la eliminación de tantos demes no es buena. Es sin duda un aspecto a mejorar en el futuro, hasta llegar al objetivo de conseguir una simulación fiable con demes de tamaño 1. El punto clave de esto será probablemente refinar la fórmula que mantiene la población constante calculando el parámetro r . Este parámetro con poblaciones pequeñas provoca subidas y bajadas bruscas del valor 'death' de cada célula del deme. Se debe buscar suavizar estos cambios para demes de poco tamaño.

Otra mejora que se aprecia útil observando estos resultados es limitar el espacio en el que el deme pueda desarrollarse, poniendo unos límites a las coordenadas (x,y,z), de manera que llegue un momento en el que el deme no pueda crecer más, y en ese momento los distintos genotipos compitan por imponerse en el territorio limitado que hay.

4.2.1. Visualización de resultados en un escenario clásico

Como ya se ha indicado anteriormente, se ha implementado una función que permite visualizar gráficamente la evolución del tumor. Se va a usar el modelo espacial con un ejemplo clásico. Este es el ejemplo propuesto por Ochs and Desai [22]. Proponen un escenario en el que ciertas combinaciones de genes mutados provocan directamente la muerte de la célula en la siguiente iteración. Esto se consigue mediante una epistasis que hace que el fitness de esa combinación sea 0.

La definición de los efectos de fitness es como la siguiente:

```

fe <- allFitnessEffects(
  data.frame(parent = c("Root", "Root", "i"),
    child = c("u", "i", "v"),
    s = c(0.1, -0.05, 0.25),
    sh = -1,
    typeDep = "MN"),
  epistasis = c("u:i" = -1, "u:v" = -1))
evalAllGenotypes(fe, order = FALSE, addwt = TRUE)
## Genotype Fitness
##1      WT      1.0000
##2       i      0.9500
##3       u      1.1000
##4       v      0.0000
##5    i , u      0.0000
##6    i , v      1.1875
##7    u , v      0.0000
##8  i , u , v      0.0000

```

Se corre el modelo con los siguientes parámetros:

```

set.seed(70)
ep2 <- oncoSimulIndiv(fe,
  mu = 1e-4,
  initSize = 1000,
  cteSize = 3000,

```

```
model = 'Constante',
sampleEvery = 0.5,
finalTime = 500,
spatialIterMax = 30,
initMutant = c("i"),
spatialModel = TRUE)
```

Los resultados finales son:

```
Final Population: 115771438 (47871 demes 70 iterations)
Genotype: u --- Population: 61723914 --- Demes: 29858 --- WinDemes: 29858
Genotype: i, v --- Population: 54047524 --- Demes: 18013 --- WinDemes: 18013
```

A continuación ejecutamos:

```
simulation3D(ep2, manual = TRUE)
```

Esto abrirá dos pantallas en las que se irá viendo como ha evolucionado la población. `manual=TRUE` indica que para avanzar en cada iteracion se debe pulsar **Enter**. Se recogen 5 capturas de la evolución del tumor:

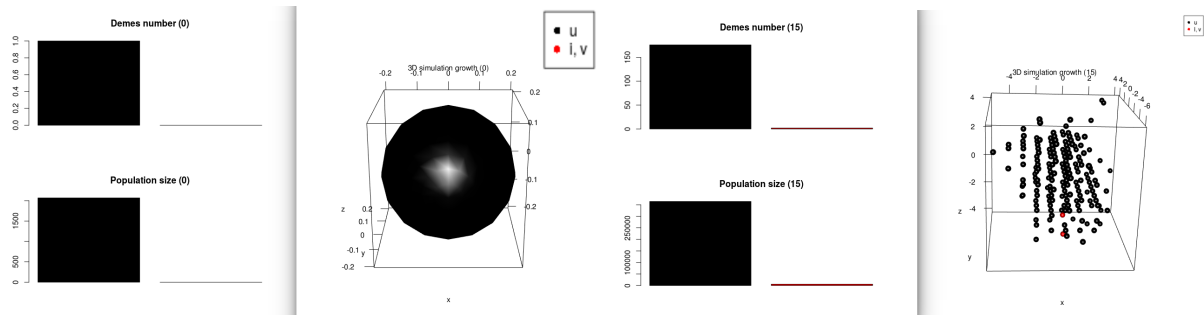


Figura 4.1: Evolución tumoral: Primeras 15 iteraciones

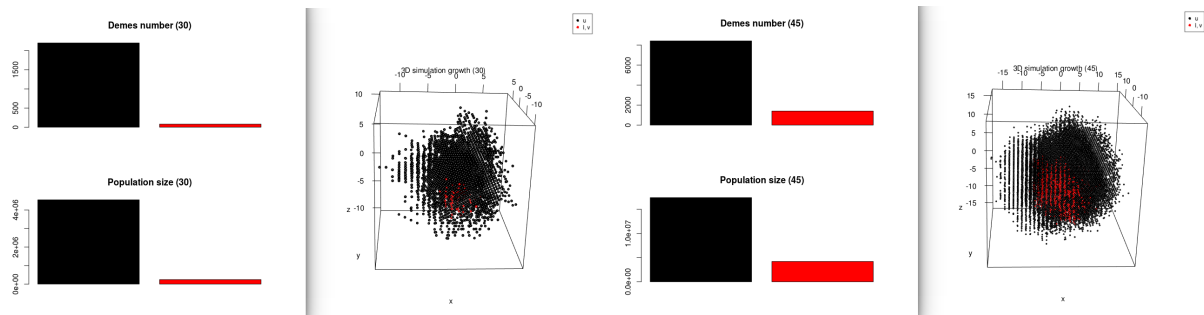


Figura 4.2: Evolución tumoral: iteraciones 15-45

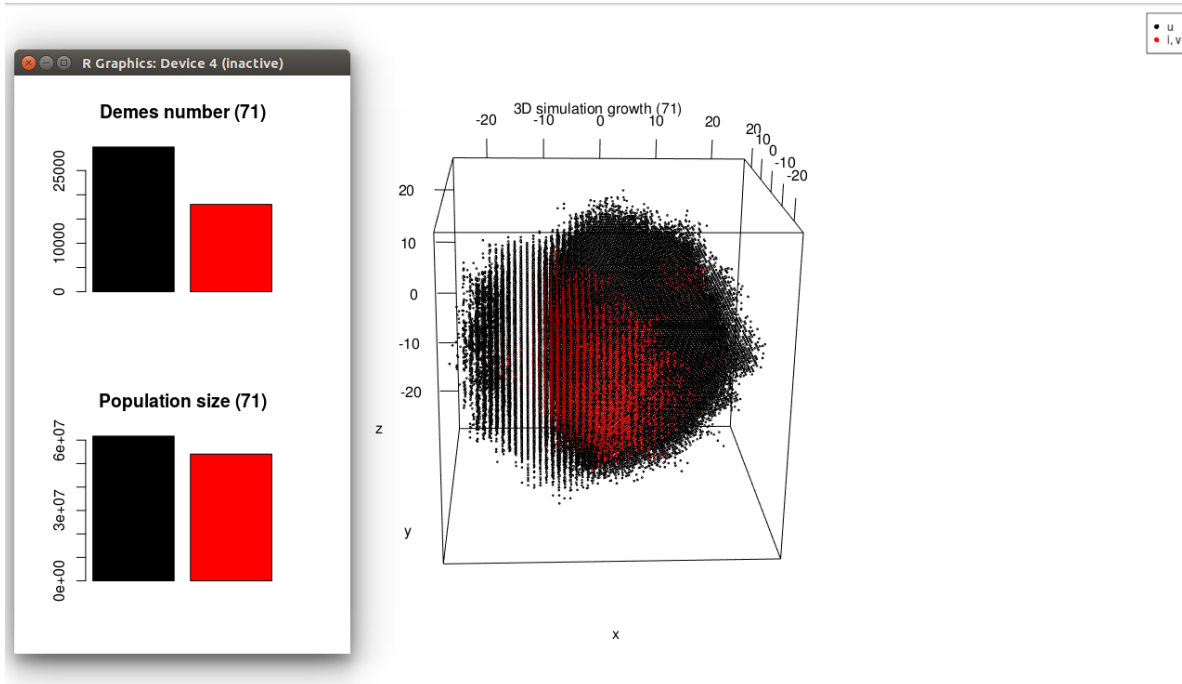


Figura 4.3: Evolución tumoral: situación final

Es interesante cómo se observa que desde que aparece la primera mutación de máximo fitness con genotipo 'i,v', esta va ganando terreno y conquistando poco a poco sus células vecinas. Además en el histograma de la izquierda se aprecia cómo representa cada vez un mayor porcentaje en el total de células del tumor, es decir, que poco a poco va conquistando el territorio y la tendencia es que siga así, por lo que en una ejecución más larga, con más iteraciones, lo hubiera conquistado por completo. Desde luego, que si se aumenta el parámetro de mutación, es de esperar que el genotipo 'i,v' tenga menos problemas en imponerse.

Cabe destacar en la visualización que aunque parezca que hay huecos entre medias de los demes esto no es así. Se representa de esta forma para que se pueda observar algo más aparte de la capa exterior.

Se contrasta ahora el parámetro `largeDistancMigrationProb` y los resultados visibles al final de la evolución. Hasta ahora este parámetro se ha mantenido en un valor muy bajo y no ha aparecido. Volviendo al genotipo con los genes 'A', 'B' y 'C' y `spatialItersMax=25`, se examina el resultado con diferentes valores:

```
set.seed(50)
ep2 <- oncoSimulIndiv(fe,
  mu = 1e-4,
  migrationProb = 1,
  largeDistanceMigrationProb = 1e-3,
  initSize = 1000,
  cteSize = 2000,
  model = 'Constante',
  sampleEvery = 0.5,
  finalTime = 200,
  spatialIterMax = 25,
  initMutant = c("A"),
  spatialModel = TRUE)
```

■ `largeDistancMigrationProb=1e-5:`

Final Population: 15657638 (8077 demes 25 iterations)

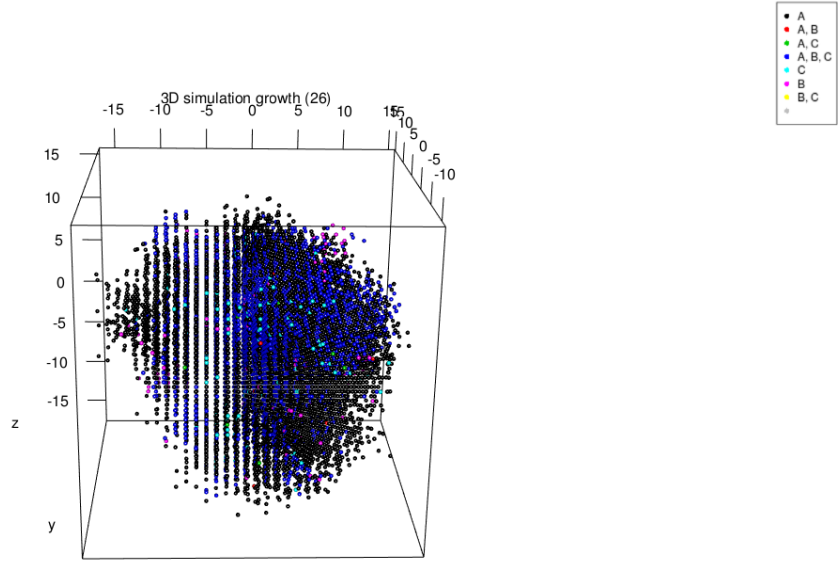


Figura 4.4: Situación final migracion larga distancia 1e-6

■ `largeDistancMigrationProb=1e-4:`

Final Population: 12993650 (6723 demes 25 iterations)

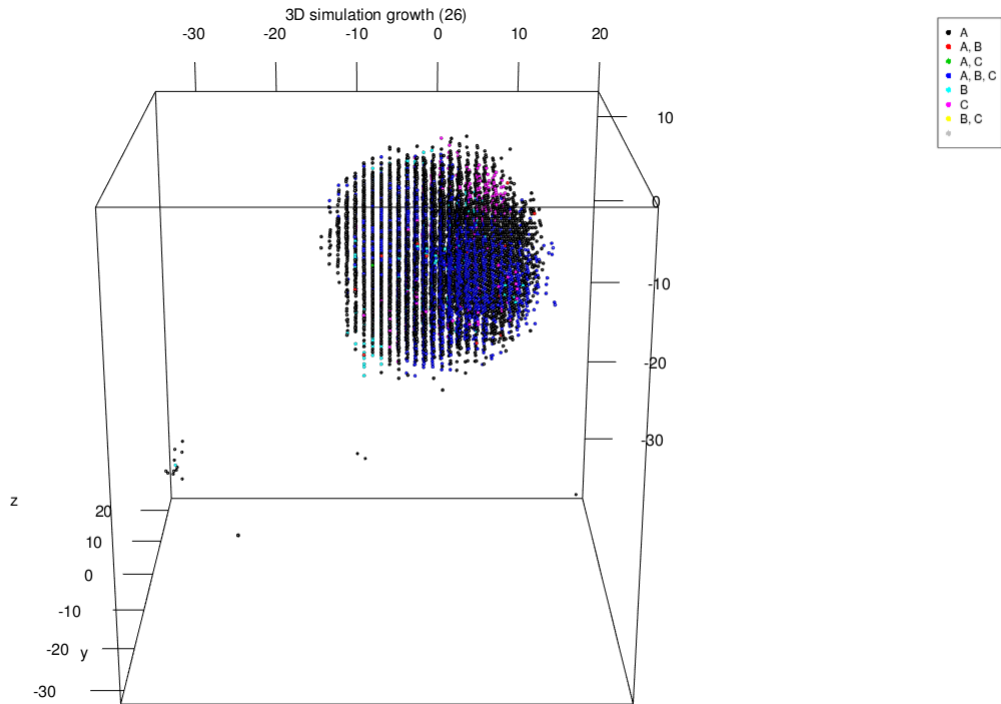


Figura 4.5: Situación final migracion larga distancia 1e-4

■ `largeDistancMigrationProb=1e-3:`

Final Population: 30604425 (15875 demes 25 iterations)

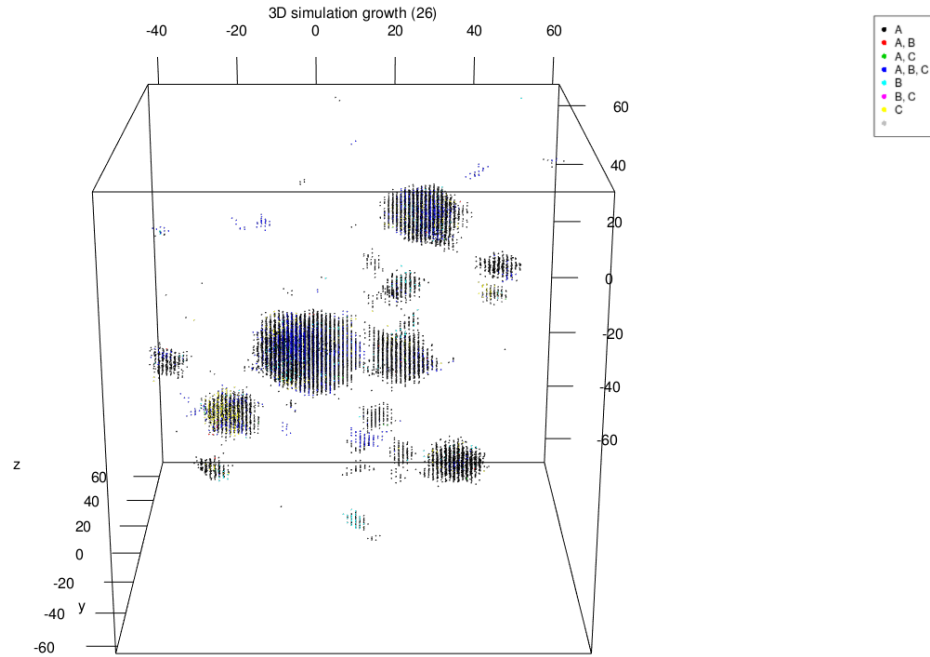


Figura 4.6: Situación final migracion larga distancia 1e-3

Cuando la probabilidad de migración a larga distancia aumenta, la capacidad de expansión del deme se dispara, dispersándose a mayor velocidad al tener también mas núcleos de proliferación en territorios con menor competencia.

4.3. Rendimiento

En esta sección se comprobará cual es el rendimiento del algoritmo. Las pruebas son realizadas en un ordenador con una memoria RAM de 5.7 GiB y un procesador Intel Core i7-4702MQ CPU @ 2.20GHz x 8.

En primer lugar se realiza una prueba de tiempos en la que se quiere comprobar cuánto tarda el algoritmo según el numero de demes que haya y el parámetro `finalTime` que es el que incide directamente en la duración de la simulación. Para ello se ejecuta

```
ep2 <- oncoSimulIndiv(fe ,
  mu = 1e-5,
  migrationProb = 1,
  largeDistanceMigrationProb = 0,
  initSize = 500,
  cteSize = 500,
  model = 'Constante',
  sampleEvery = 0.5,
  finalTime = 100,
  spatialIterMax = 20,
  initMutant = c("A"),
  spatialModel = TRUE)
```

	finalTime=500			finalTime=100		
Iteraciones	Demes	Intrademe(s)	Interdeme(s)	Demes	Intrademe(s)	Interdeme(s)
1	2	0.004028	2.4e-05	2	0.001685	2.2e-05
2	4	0.007024	2.9e-05	4	0.001492	2.3e-05
3	7	0.091508	3.4e-05	8	0.003195	3.1e-05
4	14	0.022057	3.9e-05	14	0.006901	4.4e-05
5	26	0.052801	5.5e-05	24	0.010974	5.8e-05
6	45	0.090044	7.6e-05	42	0.025172	8.3e-05
7	75	0.155422	0.00012	70	0.03253	0.000123
8	126	0.253975	0.000175	111	0.068169	0.000207
9	189	0.434812	0.000256	168	0.090279	0.000236
10	280	0.633246	0.000362	248	0.159524	0.000343
11	401	0.94408	0.000514	352	0.238355	0.000477
12	557	1.40556	0.000702	478	0.298032	0.000639
13	735	1.95181	0.000936	653	0.406701	0.000871
14	962	2.58467	0.001292	867	0.528035	0.001146
15	1235	3.75745	0.002268	1133	0.69262	0.00151
16	1567	4.74146	0.00308	1425	0.893963	0.001865
17	1953	5.64327	0.002517	1789	1.13032	0.002953
18	2381	6.73634	0.003012	2225	1.41673	0.002865
19	2907	8.14758	0.003714	2727	1.88319	0.003745
20	3475	9.94074	0.004666	3316	2.14066	0.004576

Cuadro 4.1: Resultados rendimiento de tiempos

En esta tabla se observa la influencia del parámetro finalTime, que marca completamente la duración de la simulación. También se distingue la diferencia de tiempos entre la fase intrademe e interdeme. El cuello de botella es, sin duda, y por mucha diferencia, la fase intrademe. Una mejora evidente y bastante sencilla de hacer para versiones futuras es paralelizar la fase intrademe, ya que actualmente toda la simulación se corre en un solo core e hilo de la máquina. Si bien esto en la práctica no es tan importante en el sentido en que en muchas ocasiones lo que se hace es correr muchas simulaciones distintas en paralelo del mismo escenario.

Para calcular la memoria RAM que ocupa la simulación del modelo se prueba una simulación de 50 iteraciones variando el parámetro cteSize.

```
#Prueba de rendimiento
ep2 <- oncoSimulIndiv(fe,
  mu = 1e-5,
  migrationProb = 1,
  largeDistanceMigrationProb = 0,
  initSize = 1000,
  cteSize = 1000,
  model = 'Constante',
  sampleEvery = 0.5,
  finalTime = 200,
  spatialIterMax = 50,
  initMutant = c("A"),
  spatialModel = TRUE)
```

	cteSize=1000		cteSize=100	
Iteraciones	Demes	Memoria(MB)	Demes	Memoria (MB)
1	2	<1	2	<1
2	4	<1	3	<1
7	77	<1	55	<1
8	127	<1	88	<1
9	202	<1	129	<1
10	304	<1	201	<1
11	430	<1	287	<1
12	593	<1	398	<1
13	803	<1	560	<1
14	1055	<1	768	<1
15	1331	1	1005	<1
16	1670	1	1284	1
17	2071	2	1649	1
18	2545	2	2070	1
19	3080	2	2561	2
20	3676	3	3128	2
21	4367	3	3779	2
22	5116	4	4532	3
23	6041	5	5358	4
24	6979	6	6240	5
25	8023	7	7259	6
26	9166	9	8391	7
27	10484	10	9636	8
28	11942	12	11004	10
29	13467	14	12527	12
30	15148	17	14113	14
31	16980	20	15815	17
32	18999	22	17674	20
33	21100	25	19732	23
34	23402	28	21966	27
35	25986	31	24397	32
36	28683	35	27025	36
37	31528	40	29776	41
38	34618	46	32736	46
39	37853	52	35842	51
40	41281	59	39112	57
41	44898	67	42707	65
42	48781	75	46582	74
43	52807	84	50617	84
44	57137	93	54923	95
45	61632	104	59400	106
46	66340	115	64192	118
47	71326	127	69231	130
48	76533	143	74361	142
49	82037	160	79787	162
50	87711	180	85467	185
Total	87.743.468 células	180	8.590.013 células	185

Cuadro 4.2: Resultados rendimiento de memoria

Por tanto una ejecución de 50 iteraciones en los que se han formado unos 85000 demes ocupa un 180MB de RAM, lo cual no es un valor demasiado grande. Sí se nota un crecimiento un poco exponencial, en cuando que al principio parecía que cada 10000 demes se ocupaban 10MB y la media final es de ocupar unos 21MB por cada 10000 demes. Aun así no es demasiado preocupante, ya que con un tamaño de deme estándar de 1000 células se ha conseguido simular 87 millones de células con un consumo de RAM bastante bajo.

La otra apreciación que se hace es la importancia del deme para el ahorro computacional, pues se ve que si se reduce el tamaño del mismo en una décima parte, se gasta la misma cantidad de memoria para simular la evolución de una décima parte de las células.

5

Conclusiones y líneas de trabajo futuro

A lo largo de este proyecto se ha desarrollado un modelo que simula el crecimiento de un tumor en un espacio tridimensional. Se han dado las bases matemáticas del mismo y se ha explicado el algoritmo informático que lo formaliza. Además se ha integrado en el paquete *OncoSimulR*, empleando y extendiendo sus funcionalidades y explicando su uso.

He utilizado mis conocimientos en programación en C++ y R y los he ampliado; también he visto la utilidad y las ventajas de utilizar los conocimientos de Matemáticas e Informática en otras ramas, como la Genética o la Medicina. He aplicado también mis conocimientos en modelización para investigar en un campo que está creciendo recientemente, como así lo muestran los papers de Waclaw (2015) [12] o Iwasaki(2017) [17].

Además, he aprendido que el resultado final de las simulaciones no es tan trivial como suponer que se va a imponer el genotipo más ventajoso. Cómo se ha visto en la sección anterior, hay situaciones en que la llegada al genotipo más ventajoso es complicada y en poblaciones de células pequeñas o de baja probabilidad de mutación. También se ha observado que en un escenario con los mismos efectos de fitness y los mismos posibles genotipos, una pequeña variación en los tres parámetros principales (tiempo, población y probabilidad de mutación) puede producir desenlaces diferentes.

Considero que el modelo implementado no tiene nada que envidiar a ninguno de los que me han hecho aprender, pero por supuesto, sí tiene muchos posibles aspectos a mejorar, algunos de los cuales se han ido mencionando a lo largo de esta memoria y que recopilo a continuación:

A corto plazo:

- Permitir al usuario acotar el espacio en el que el tumor puede crecer. Esto puede ser útil para simular algunas situaciones de evolución tumoral muy concretas.
- Afinar el modelo para que soporte simulaciones con demes de tamaño 1, es decir, de una célula.
- Ampliar la información de salida que ofrece el modelo para poder calcular fácilmente distintos valores estadísticos o zonas de concentración de algunos genotipos.
- En cuanto al rendimiento, paralelizar la fase intrademe para conseguir ejecuciones más rápidas.

A largo plazo, y con un mayor estudio de la utilidad:

- Permitir al usuario personalizar la cantidad de células que hay inicialmente en cada zona del espacio, pudiendo hacer un mapa del estado inicial del espacio tridimensional.
- Permitir al usuario la posibilidad de añadir factores microambientales externos, que diferentes zonas del espacio presenten unas características concretas que afecten al crecimiento del tumor, y que el fitness de los genotipos cambie con el tiempo.
- Añadir relaciones de interacción directa entre determinados tipos de células. Por ejemplo, que la evolución de algunas células cambie según si hay células de un determinado tipo que se sitúan cerca de ellas.

Para finalizar, y como opinión personal, quiero valorar la utilidad de estos modelos. En un principio a mi me cuesta creer que esto puede llegar a ser de utilidad para investigar la evolución de los tumores en el cuerpo humano, pero tras estar bastante tiempo investigando sobre el tema me di cuenta de que se está avanzando a un ritmo importante en esta línea, ya que se ha pasado en pocos años de simular crecimientos adimensionalmente a crear una malla tridimensional donde las células compiten por un espacio físico. Por tanto, cada vez veo más probable que en un futuro no muy lejano se puedan crear modelos elaborados que simulen con mucha mayor precisión el desarrollo de los tumores dentro del cuerpo humano.

Bibliografía

- [1] Informe anual a la nación: la mortalidad por cáncer en general sigue descendiendo, la mortalidad por cáncer de próstata se ha estabilizado. *Informe Anual de la Nación*, 2018. <https://www.cancer.gov/espanol/noticias/comunicados-de-prensa/2018/informe-anual-al-pais-2018>. 1.1
- [2] Philipp M. Altrock, Li L. Liu, and Franziska Michor. The mathematics of cancer: integrating quantitative models. *Nature*, 2015. <http://www.ncbi.nlm.nih.gov/pubmed/26597528>. 2.1
- [3] Niko Beerenwinkel, Roland F. Schwarz, Moritz Gerstung, and Florian Markowetz. Cancer evolution: Mathematical models and computational inference. *Systematic Biology*, 2012. <http://www.ncbi.nlm.nih.gov/pubmed/25293804>. 2.1
- [4] R.B. Campbell. A logistic branching process for population genetics. *Journal of Theoretical Biology*, 2002. <http://www.ncbi.nlm.nih.gov/pubmed/14575653>. 2.1
- [5] Wonpyong Gill. Modified fixation probability in multiple alleles models in the asymmetric sharply-peaked landscape. *Journal of the Korean Physical Society*, 2009. https://www.researchgate.net/publication/243586892_Modified_Fixation_Probability_in_Multiple_Alleles_Models_in_theAsymmetric_Sharpely-Peaked_Landscape. 2.1
- [6] John H. Gillespie. Substitution processes in molecular evolution. i. uniform and clustered substitutions in a haploid model. *Genetics*, 1993. <http://www.ncbi.nlm.nih.gov/pubmed/8349119>. 2.1
- [7] Benjamin H. Good, Igor M. Rouzine, Daniel J. Balick, Oskar Hallatschek, and Michael M. Desai. Distribution of fixed beneficial mutations and the rate of adaptation in asexual populations. *PNAS*, 2012. <http://www.pnas.org/content/109/13/4950>. 2.1
- [8] Christopher J. R. Illingworth and Mustonen Ville. Distinguishing driver and passenger mutations in an evolutionary history categorized by interference. *Genetics*, 2011. <http://www.ncbi.nlm.nih.gov/pubmed/21900272>. 2.1
- [9] Michael A. Gibson and Bruck Jehoshua. Efficient exact stochastic simulation of chemical systems with many species and many channels. *American Chemical Society*, 2000. <http://pubs.acs.org/doi/abs/10.1021/jp993732q>. 2.1, 2.3.1
- [10] William H. Mather, Jeff Hasty, and Lev S. Tsimming. Fast stochastic algorithm for simulating evolutionary population dynamics. *Bioinformatics*, 2012. <http://www.ncbi.nlm.nih.gov/pubmed/22437850>. 2.1, 2.3.1, 2.3.1, 2, 3.1.4
- [11] Erik A. Martens, Rumen Kostadinov, and Carlo C. Maley. Spatial structure increases the waiting time for cancer. *New Journal of Physics*, 2011. <http://arxiv.org/abs/1106.3005>. 2.1

- [12] Bartłomiej Waclaw, Ivana Bozic, Meredith E. Pittman, Ralph H. Hruban, Bert Vogelstein, and Martin A. Nowak. A spatial model predicts that dispersal and cell turnover limit intratumour heterogeneity. *Nature*, 2015. <http://www.ncbi.nlm.nih.gov/pubmed/26308893>. 2.1, 2.2.1, 2.1, 3.1.3, 4.2, 5
- [13] A. Sottoriva, Z. Hu, H. Kang, M.P. Salomon, J. Zhao, P. Marjoram, K. Siegmund, M.F. Press, T.A. Graham, Z. Ma, D. Shibata, and C. Curtis. A big bang model of human colorectal tumor growth. *Genetics*, 2015. <https://www.nature.com/articles/ng.3214>. 2.2.2, 3.1.3
- [14] R. Sun, Z. Hu, A. Sottoriva, T.A. Graham, A. Harpak, Z. Ma, J.M. Fischer, D. Shibata, and C. Curtis. Between-regin genetic divergence reflects the mode and tempo of tumor evolution. *Genetics*, 2017. <https://www.ncbi.nlm.nih.gov/pubmed/28581503>. 2.2.3
- [15] M.C Lloyd. Darwinian dynamics of intratumoral heterogeneity: not solely random mutations but also variable environmental selection forces. *Genetics*, 2016. <https://www.ncbi.nlm.nih.gov/pubmed/27009166>. 2.2.3
- [16] J. Poleszczuk, P. Hahnfeldt, and H. Enderling. Evolution and phenotypic selection of cancer stem cells. *PLOS Computational biology*, 2015. <http://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004025>. 2.2.4
- [17] W.M. Iwasaki and H. Innan. Simulation framework for generating intratumor heterogeneity patterns in a cancer cell population. *PLOS one*, 2017. <http://journals.plos.org/plosone/article?id=10.1371/journal.pone.0184229>. 2.2.4, 3.1.3, 3.1, 5
- [18] R. Uchi, Y. Takahashi, A. Niida, T. Shimamura, H. Hirata, and K. Sugimachi. Integrated multiregional analysis proposing a new model of colorectal cancer evolution. *PLOS Genetics*, 2015. <http://journals.plos.org/plosgenetics/article?id=10.1371/journal.pgen.1005778>. 2.2.4
- [19] R Noble. The mode and predictability of intra-tumour evolution. *presentación no publicada*. 2.2.4, 2.1
- [20] Ramón Díaz Uriarte. Oncosimulr: forward genetic simulation in asexual populations with arbitrary epistatic interactions and a focus on modeling tumor progression. *BMC Bioinformatics*, 2016. <https://github.com/rdiaz02/OncoSimul>. 2.3
- [21] Sean H Rice. Evolutionary theory: Mathematical and conceptual foundations. *Libro*. 4.2
- [22] Ian E Ochs and M Michael Desai. The competition between simple and complex evolutionary trajectories in asexual populations. *BMC Evolutionary Biology*, 2015. <http://www.biomedcentral.com/1471-2148/15/55>, doi:10.1186/s12862-015-0334-0. 4.2.1